

When Every Row Becomes a Curve: Using Andrews Transforms to Turn Multivariate Wine Data into Actionable Quality Intelligence

Outlier detection, clustering, hypothesis testing, and process monitoring — all from curves that preserve every relationship in your data

Simon Müller

```
library(fdars)
library(ggplot2)
library(dplyr)
library(patchwork)
library(knitr)
theme_set(theme_minimal(base_size = 13))
set.seed(42)
```

The Problem with Tables

A quality-control analyst at a winery sits down to review the latest batch of 178 wines. Each sample has been tested for 13 chemical properties — alcohol content, malic acid, ash, alkalinity, magnesium, phenols, flavanoids, nonflavanoid phenols, proanthocyanins, color intensity, hue, OD280/OD315 dilution ratio, and proline. That's $178 \times 13 = \mathbf{2,314 \text{ numbers}}$. The analyst needs to answer simple questions: *Are there any anomalous wines? Do the three cultivars (Barolo, Grignolino, Barbera) look chemically distinct? Which chemicals matter most for telling them apart?*

Behind these questions sits a business reality. The winery needs to:

1. **Detect production anomalies** before a defective batch ships. A single mislabeled or adulterated wine can trigger a recall.
2. **Validate cultivar identity** for denomination-of-origin certification. The regulator wants proof, not assertions.
3. **Reduce testing costs** by identifying which of the 13 assays are truly necessary and which are redundant.

4. **Establish ongoing monitoring** so that *future* batches can be checked against a validated reference profile — not re-analyzed from scratch every time.

Standard practice tackles these one at a time: a spreadsheet for anomaly screening, a MANOVA for group differences, a PCA for variable reduction, a separate set of per-variable control charts for monitoring. Each tool uses its own distance metric, its own assumptions, and its own output format. Nothing connects them.

This article demonstrates a different approach: **transform the 13 chemical measurements into Andrews curves**, and then apply a unified pipeline of functional data analysis (FDA) methods. Each step delivers a concrete business outcome:

Step	Method	Business Outcome
Visualize	Andrews curves	See each wine as a single chemical fingerprint
Detect	Depth, outliergram, MS-plot	Triage anomalies by <i>type</i> (magnitude, shape, mislabel)
Validate	Clustering, permutation test	Prove cultivars are chemically distinct (p-value)
Optimize	FPCA, variable importance	Identify which 5 of 13 chemicals to keep testing
Monitor	Functional boxplot, tolerance bands	Check new batches against a validated specification

The key guarantee: Andrews curves preserve Euclidean distances exactly (up to a constant $\sqrt{\pi}$), so every conclusion translates back to the original chemical units. Nothing is lost in the transformation.

Let's start with the data and see why a spreadsheet isn't enough.

```
# UCI Wine dataset: 178 wines, 3 cultivars, 13 chemical measurements
wine_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data"
col_names <- c("Cultivar", "Alcohol", "MalicAcid", "Ash", "Alkalinity",
               "Magnesium", "Phenols", "Flavanoids", "NonflavPhenols",
               "Proanthocyanins", "ColorIntensity", "Hue",
               "OD280_OD315", "Proline")
wine <- read.csv(wine_url, header = FALSE, col.names = col_names)

# Use real cultivar names
cultivar <- factor(wine$Cultivar,
                   levels = 1:3,
                   labels = c("Barolo", "Grignolino", "Barbera"))
```

```
# Standardize the 13 chemical variables
X <- scale(as.matrix(wine[, -1]))
chem_names <- colnames(wine)[-1]

cat(nrow(wine), "wines,", ncol(X), "chemicals,",
    nlevels(cultivar), "cultivars\n")
```

178 wines, 13 chemicals, 3 cultivars

```
df_box <- data.frame(X) |>
  mutate(Cultivar = cultivar) |>
  tidyr::pivot_longer(-Cultivar, names_to = "Chemical", values_to = "Value")

ggplot(df_box, aes(x = Cultivar, y = Value, fill = Cultivar)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.8) +
  facet_wrap(~ Chemical, scales = "free_y", ncol = 4) +
  scale_fill_manual(values = c("Barolo" = "#8B0000",
                              "Grignolino" = "#DAA520",
                              "Barbera" = "#2E8B57")) +
  labs(x = NULL, y = "Standardized Value") +
  theme(legend.position = "bottom",
        strip.text = element_text(size = 9))
```

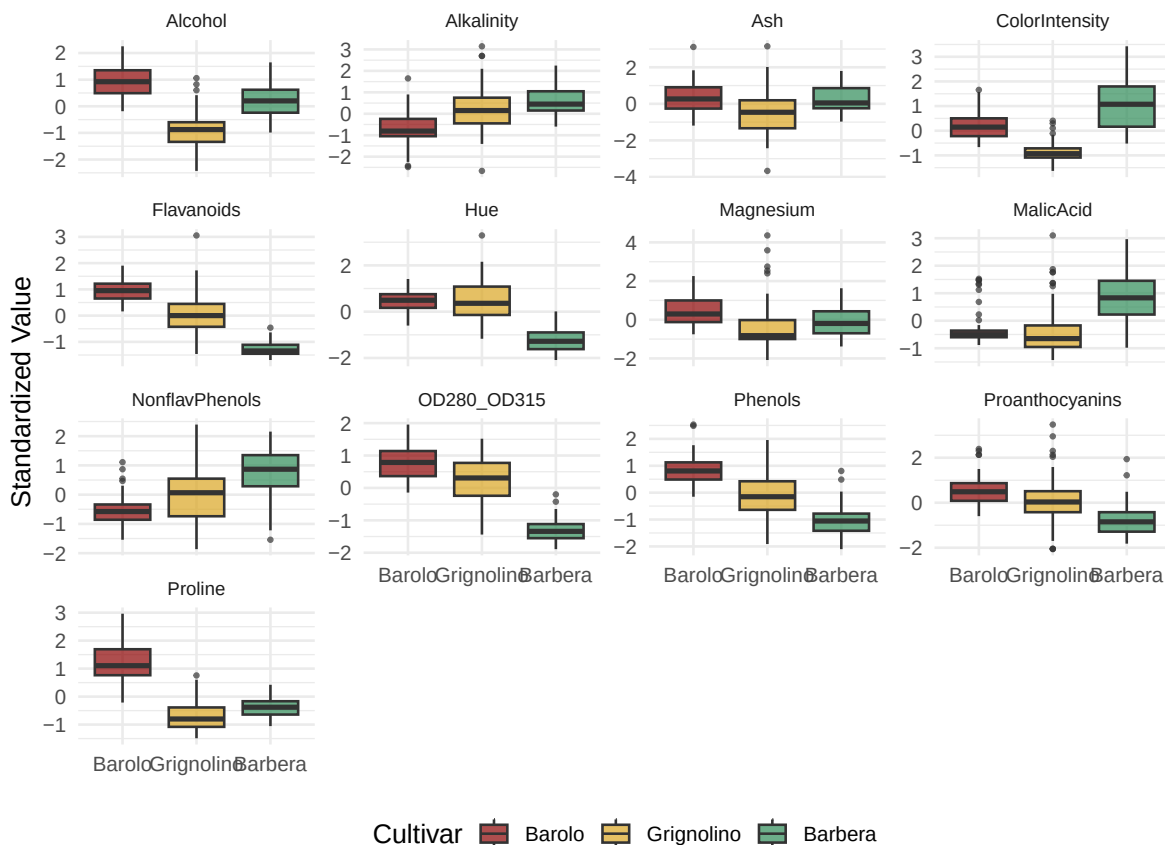


Figure 1: Standardized chemical profiles by cultivar — some variables separate groups, most overlap.

Some variables (Flavanoids, Proline, Color Intensity) clearly separate cultivars. Others (Ash, Magnesium) overlap heavily. But you can't *see a wine* here — you see 13 disconnected box-and-whisker slices. Every decision about blending, pricing, or fraud detection requires a **holistic view** of each wine's chemical fingerprint. That's where Andrews curves come in.

Turning Rows into Curves

The **Andrews transformation** (Andrews, 1972) maps each p -dimensional observation $\mathbf{x} = (x_1, \dots, x_p)$ to a curve on $[-\pi, \pi]$ using a Fourier expansion:

$$f_{\mathbf{x}}(t) = \frac{x_1}{\sqrt{2}} + x_2 \sin(t) + x_3 \cos(t) + x_4 \sin(2t) + x_5 \cos(2t) + \dots$$

The key guarantee: L^2 distance between curves equals $\sqrt{\pi}$ times the Euclidean distance between the original vectors. Nothing is lost. Nothing is distorted.

```
andrews_transform <- function(X, m = 200) {
  if (is.data.frame(X)) X <- as.matrix(X)
  n <- nrow(X)
  p <- ncol(X)
  t_grid <- seq(-pi, pi, length.out = m)

  # Build Fourier basis matrix [m x p]
  basis <- matrix(0, m, p)
  basis[, 1] <- 1 / sqrt(2)
  for (j in 2:p) {
    k <- ceiling((j - 1) / 2)
    if (j %% 2 == 0) {
      basis[, j] <- sin(k * t_grid)
    } else {
      basis[, j] <- cos(k * t_grid)
    }
  }

  fdata(X %*% t(basis), argvals = t_grid)
}

fd_wine <- andrews_transform(X)
cat("Andrews curves:", nrow(fd_wine$data), "observations,",
    ncol(fd_wine$data), "grid points\n")
```

Andrews curves: 178 observations, 200 grid points

```
n <- nrow(fd_wine$data)
m <- ncol(fd_wine$data)
t_grid <- fd_wine$argvals

df_curves <- data.frame(
  t = rep(t_grid, n),
  value = as.vector(t(fd_wine$data)),
  curve = rep(1:n, each = m),
  Cultivar = rep(cultivar, each = m)
)

ggplot(df_curves, aes(x = t, y = value, group = curve, color = Cultivar)) +
```

```
geom_line(alpha = 0.35, linewidth = 0.4) +
scale_color_manual(values = c("Barolo" = "#8B0000",
                              "Grignolino" = "#DAA520",
                              "Barbera" = "#2E8B57")) +
labs(title = "Andrews Curves of Wine Chemical Profiles",
     x = expression(t), y = expression(f[x](t))) +
theme(legend.position = "bottom")
```

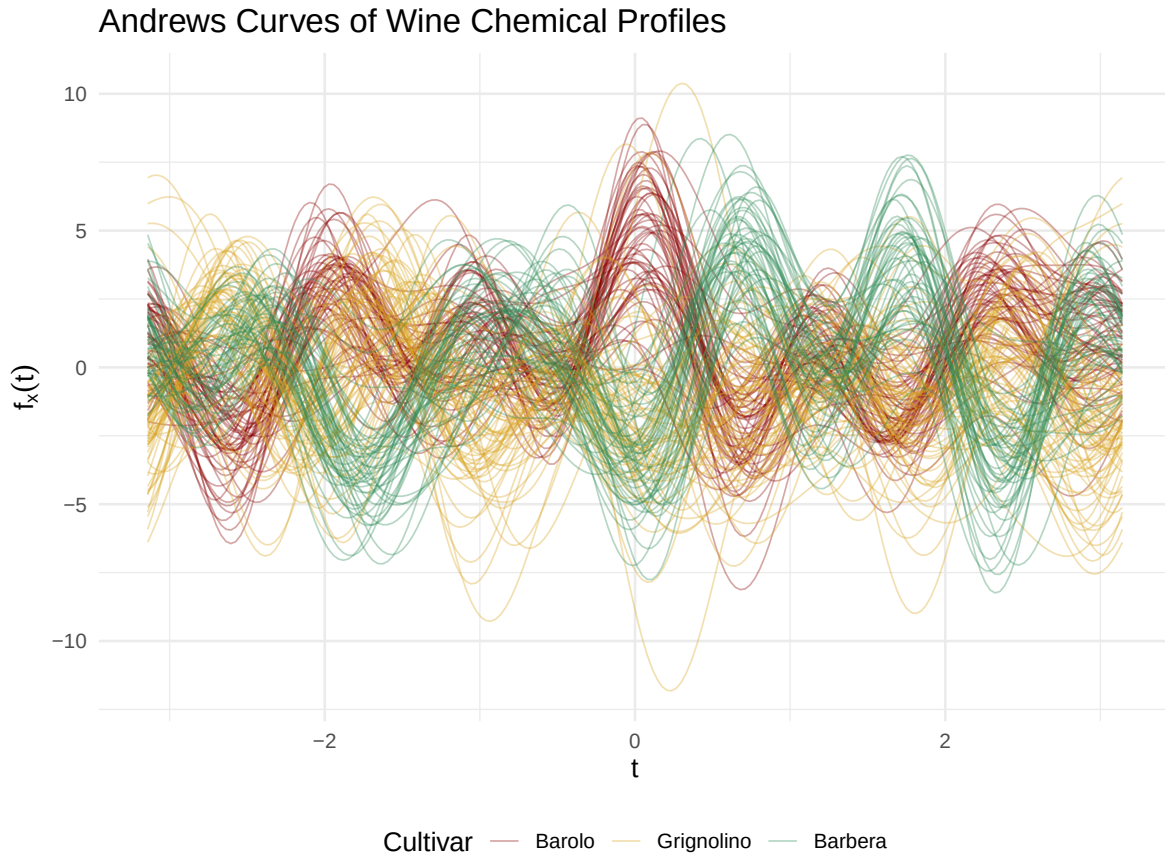


Figure 2: 178 wines as Andrews curves — each curve is a chemical fingerprint. Cultivar separation is immediately visible.

Now each wine has a **signature** — a single visual object that encodes all 13 measurements. A quality manager can glance at a curve and compare it to the cultivar’s typical profile. Wines that “look different” are immediately suspicious. This is the multivariate equivalent of a chromatography trace — one picture captures the whole chemical identity.

Proving the Bridge Works

Pretty pictures are nice, but for regulatory or audit contexts, we need a **mathematical guarantee**. The Andrews distance preservation theorem says:

$$\|f_{\mathbf{x}} - f_{\mathbf{y}}\|_{L^2} = \sqrt{\pi} \cdot \|\mathbf{x} - \mathbf{y}\|_2$$

Let's verify this on all $\binom{178}{2} = 15,753$ pairwise distances.

```
# Compute pairwise distances in both domains
dist_andrews <- metric.lp(fd_wine)
dist_euclid <- as.matrix(dist(X))

# Extract upper triangle
idx_upper <- upper.tri(dist_andrews)
d_a <- dist_andrews[idx_upper]
d_e <- dist_euclid[idx_upper]

# Filter zero-distance pairs (if any duplicates)
nonzero <- d_e > 1e-10
ratio <- d_a[nonzero] / d_e[nonzero]

cat(sprintf("Distance ratio (Andrews / Euclidean):\n"))
```

Distance ratio (Andrews / Euclidean):

```
cat(sprintf("  Mean:      %.4f\n", mean(ratio)))
```

Mean: 1.7725

```
cat(sprintf("  Median:    %.4f\n", median(ratio)))
```

Median: 1.7725

```
cat(sprintf("  SD:        %.2e\n", sd(ratio)))
```

SD: 4.01e-16

```
cat(sprintf("  sqrt(pi): %.4f\n", sqrt(pi)))
```

```
sqrt(pi): 1.7725
```

```
df_dist <- data.frame(euclidean = d_e[nonzero], andrews = d_a[nonzero])

ggplot(df_dist, aes(x = euclidean, y = andrews)) +
  geom_point(alpha = 0.08, size = 0.5, color = "steelblue") +
  geom_abline(slope = sqrt(pi), intercept = 0, color = "red", linewidth = 1) +
  labs(title = "Andrews L2 Distance vs Euclidean Distance",
       subtitle = sprintf("Red line: slope =  $\sqrt{\pi}$  %.4f", sqrt(pi)),
       x = "Euclidean Distance (standardized data)",
       y = "Andrews L2 Distance") +
  coord_equal()
```

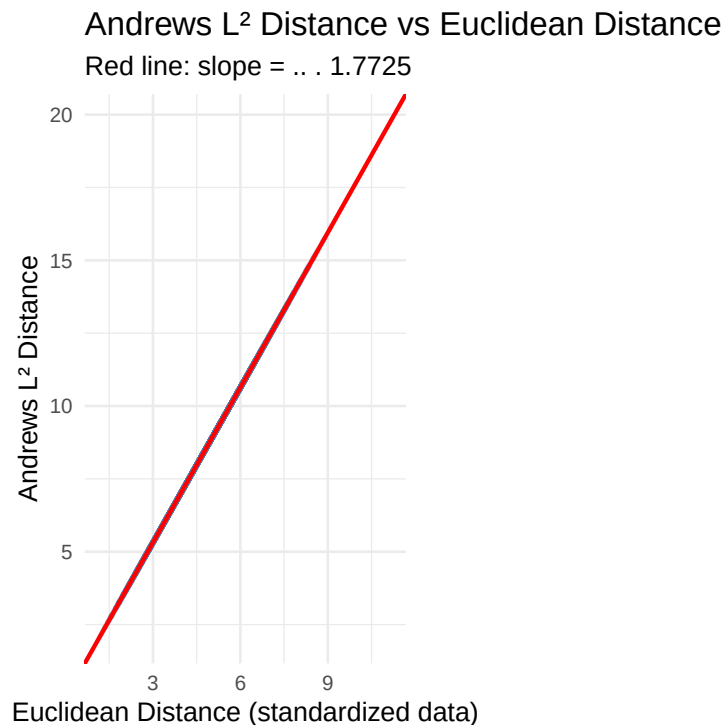


Figure 3: Every pairwise distance falls exactly on the $\sqrt{\pi}$ line — the transformation is lossless.

The ratio is $\sqrt{\pi} \approx 1.7725$ to machine precision. Any conclusion drawn from the curves — outliers, clusters, similarities — translates *exactly* back to the original 13 chemical measurements.

For regulatory or audit contexts, this mathematical proof is essential: you can report findings in chemical units, not abstract curve distances.

Finding the Unusual Wines

In wine production, “unusual” can mean a quality problem, a labeling error, or outright fraud. A standard approach might flag individual chemicals that fall outside a reference range — but that misses wines whose *combination* of values is strange, even when no single measurement is extreme. Because Andrews curves encode *all 13 chemicals simultaneously*, outlier detection on curves catches multivariate anomalies that univariate checks miss.

We apply three complementary methods. Each asks a different question about “unusualness,” and each gives the quality controller a different kind of actionable insight.

Method 1: Depth-Ponderation — “How far from the crowd?”

Think of *depth* as the opposite of outlyingness. The most “typical” wine — the one whose chemical profile sits squarely in the middle of the population — has the highest depth. Wines at the fringes have low depth. The depth-ponderation method (`outliers.depth.pond()`) stabilizes this measurement using **bootstrap resampling**: it repeatedly draws random subsets of the 178 wines, computes depth each time, and averages the results. This smoothing prevents a single unlucky sample from distorting the picture. Wines whose averaged depth falls below a threshold are flagged.

```
set.seed(123)
out_pond <- outliers.depth.pond(fd_wine, nb = 500)
cat("Depth-ponderation outliers:", sort(out_pond$outliers), "\n")
```

Depth-ponderation outliers: 60 70 74 79 96 97 111 122 159

```
cat("Cultivars:", as.character(cultivar[out_pond$outliers]), "\n")
```

Cultivars: Grignolino Grignolino Grignolino Grignolino Grignolino Grignolino Grignolino Grignolino Grignolino

```
is_pond_outlier <- rep(FALSE, n)
is_pond_outlier[out_pond$outliers] <- TRUE

ggplot() +
  geom_line(data = filter(df_curves, !rep(is_pond_outlier, each = m)),
    aes(x = t, y = value, group = curve),
```

```

    color = "gray70", alpha = 0.3, linewidth = 0.3) +
  geom_line(data = filter(df_curves, rep(is_pond_outlier, each = m)),
    aes(x = t, y = value, group = curve, color = Cultivar),
    linewidth = 0.7, alpha = 0.8) +
  scale_color_manual(values = c("Barolo" = "#8B0000", "Grignolino" = "#DAA520",
    "Barbera" = "#2E8B57")) +
  labs(title = "Depth-Ponderation Outliers",
    subtitle = sprintf("%d wines flagged - curves far from the population center",
      length(out_pond$outliers)),
    x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

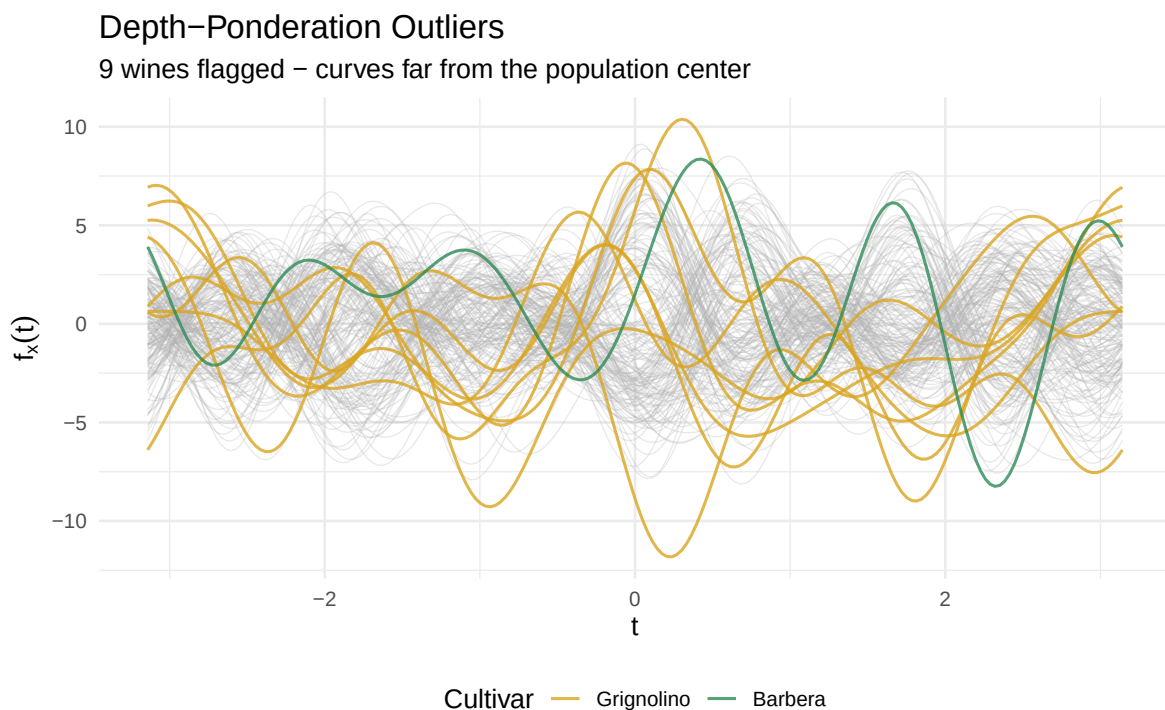


Figure 4: Depth-ponderation outliers (red) sit at the fringes of the curve bundle — chemical profiles unlike any typical wine.

What makes these wines unusual? Let's look at which chemicals push them to the fringes:

```

# For each flagged wine, identify chemicals with |z-score| > 2
extreme_list <- lapply(sort(out_pond$outliers), function(i) {
  extreme_idx <- which(abs(X[i, ]) > 2)

```

```

if (length(extreme_idx) > 0) {
  chems <- paste(sprintf("%s (z=%.1f)", colnames(wine)[-1][extreme_idx],
                        X[i, extreme_idx]), collapse = ", ")
} else {
  chems <- "no individual extreme - unusual *combination*"
}
data.frame(Wine = i, Cultivar = as.character(cultivar[i]),
           `Extreme Chemicals` = chems, check.names = FALSE)
})
kable(bind_rows(extreme_list),
      caption = "What makes each depth-ponderation outlier unusual ( $|z| > 2$ )")

```

Table 2: What makes each depth-ponderation outlier unusual ($|z| > 2$)

Wine	Cultivar	Extreme Chemicals
60	Grignolino	Ash (z=-3.7), Alkalinity (z=-2.7), Proanthocyanins (z=-2.0)
70	Grignolino	Ash (z=-2.2), Magnesium (z=3.6)
74	Grignolino	Alkalinity (z=3.1), Magnesium (z=2.7)
79	Grignolino	Magnesium (z=2.5), Proanthocyanins (z=2.0)
96	Grignolino	Magnesium (z=4.4), Proanthocyanins (z=3.0)
97	Grignolino	Magnesium (z=2.4)
111	Grignolino	Proanthocyanins (z=3.5)
122	Grignolino	Ash (z=3.1), Alkalinity (z=2.7), Flavanoids (z=3.1)
159	Barbera	ColorIntensity (z=3.4)

Notice that several wines have no individual extreme value — they’re outliers because of an unusual *combination* of chemicals. This is exactly the kind of anomaly that a column-by-column check would miss.

Method 2: Magnitude-Shape Plot — “Unusual *how*?”

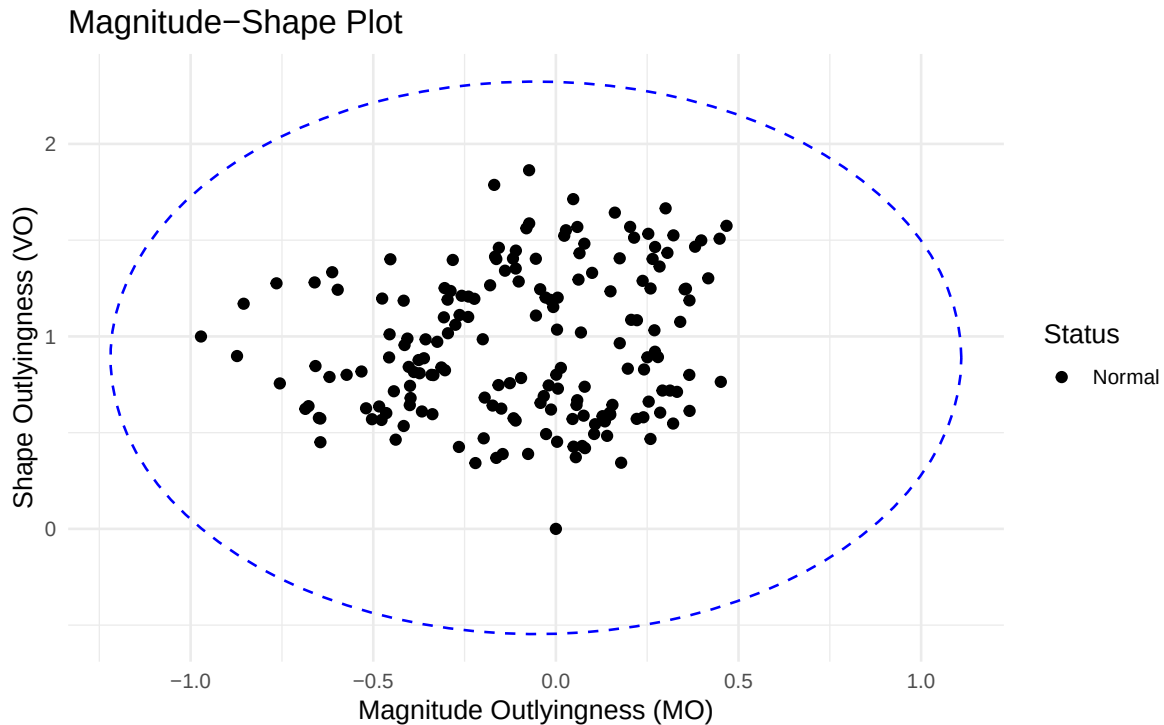
The depth-ponderation method tells you *that* a wine is unusual, but not *how*. The magnitude-shape decomposition (`magnitudeshape()`) splits outlyingness into two independent components:

- **Magnitude outlyingness (MO)** measures whether a wine’s curve is systematically shifted up or down relative to the median wine. In chemical terms: are *all* values consistently high or consistently low? A high MO suggests a global issue — perhaps a measurement calibration error, a different vintage mixed in, or dilution/concentration.

- **Variability outlyingness (VO)** measures whether the *shape* of a wine’s curve deviates from the population’s typical shape, regardless of its overall level. In chemical terms: are the *ratios* between chemicals unusual? A high VO suggests a process anomaly — perhaps a fermentation gone wrong, soil contamination, or blending with a different cultivar.

The method plots every wine in this two-dimensional (MO, VO) space and draws a χ^2 -based cutoff ellipse. Wines outside the ellipse are flagged.

```
ms <- magnitudeshape(fd_wine)
plot(ms)
```



With the default 99.3% cutoff, no wines are flagged here. This is informative, not disappointing — it tells us that the outliers found by depth-ponderation are **not extreme in the magnitude-shape sense**. They sit at the fringes of their cultivar group, but their chemical *ratios* are still within the normal range. For a quality controller, this is reassuring: these wines are unusual but not *suspiciously* unusual. If the MS-plot *did* flag a wine, that would be a stronger alarm — possibly indicating adulteration or a process failure that distorts the fundamental chemical fingerprint.

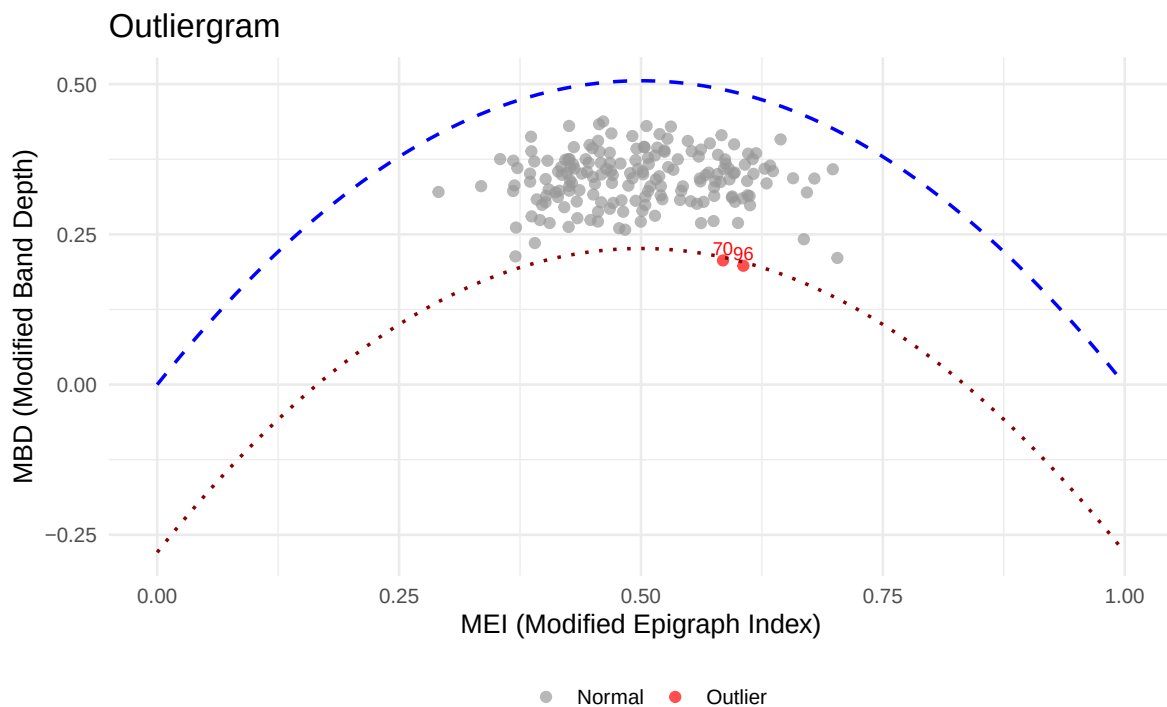
Method 3: Outliergram — “Unusual in rank structure?”

The outliergram (`outliergram()`) takes a different angle entirely. It computes two rank-based statistics for each wine:

- **Modified Epigraph Index (MEI)**: what proportion of the time does this wine’s curve sit *above* other curves? A wine with MEI near 0.5 is average in its vertical position; a wine near 0 or 1 is persistently low or high.
- **Modified Band Depth (MBD)**: how often does this wine’s curve fall *within the band* formed by pairs of other curves? High MBD means the wine has a typical profile; low MBD means it tends to cross or escape the population envelope.

For a well-behaved population, MEI and MBD follow a tight parabolic relationship: wines that are vertically central (MEI = 0.5) should also be deeply enclosed (high MBD). The outliergram flags wines that deviate from this expected relationship — wines whose MBD is *lower than expected* given their MEI. These are **shape outliers**: wines that weave in and out of the population bundle in an unusual way.

```
og <- outliergram(fd_wine)
plot(og)
```



```

if (length(og$outliers) > 0) {
  cat("Outliergram outliers:", sort(og$outliers), "\n")
  cat("Cultivars:", as.character(cultivar[og$outliers]), "\n")
  for (i in sort(og$outliers)) {
    extreme_idx <- which(abs(X[i, ]) > 2)
    if (length(extreme_idx) > 0) {
      cat(sprintf("  Wine %d: %s\n", i,
                  paste(sprintf("%s (z=%.1f)", colnames(wine)[-1][extreme_idx],
                                X[i, extreme_idx]), collapse = ", ")))
    }
  }
}

```

```

Outliergram outliers: 70 96
Cultivars: Grignolino Grignolino
  Wine 70: Ash (z=-2.2), Magnesium (z=3.6)
  Wine 96: Magnesium (z=4.4), Proanthocyanins (z=3.0)

```

The outliergram flags the most severe shape anomalies — wines whose internal chemical relationships are unusual even though their overall levels may be plausible. For a quality controller, a shape outlier warrants a different investigation than a magnitude outlier: you wouldn't re-calibrate the instrument; you'd investigate the vineyard, the fermentation, or the supply chain.

Cross-Referencing: The Confidence Triage

No single outlier detection method is perfect. Each trades off sensitivity against specificity, and each is blind to certain kinds of anomaly. The real power comes from **cross-referencing**: wines flagged by multiple methods are the highest-priority cases for investigation.

```

# Collect flags per observation
outlier_flags <- data.frame(
  Wine = 1:n,
  Cultivar = cultivar,
  DepthPond = 1:n %in% out_pond$outliers,
  MagnShape = 1:n %in% ms$outliers,
  Outliergram = 1:n %in% og$outliers
)
outlier_flags$n_methods <- rowSums(outlier_flags[, 3:5])

# Show wines flagged by at least one method

```

```
flagged <- outlier_flags |> filter(n_methods >= 1) |> arrange(desc(n_methods))

cat(nrow(flagged), "wines flagged by at least one method,",
    sum(flagged$n_methods >= 2), "by two or more\n\n")
```

9 wines flagged by at least one method, 2 by two or more

```
# Show flagged wines with their original chemical values
flagged_detail <- flagged |>
  mutate(across(c(DepthPond, MagnShape, Outliergram),
    ~ ifelse(., " ", ""))) |>
  left_join(
    data.frame(Wine = 1:n, round(wine[, -1], 1)),
    by = "Wine"
  )

kable(flagged_detail |> select(Wine, Cultivar, DepthPond, MagnShape,
  Outliergram, n_methods,
  Alcohol, Flavanoids, ColorIntensity, Proline),
  caption = "Flagged wines with key chemical values")
```

Table 3: Flagged wines with key chemical values

Wine	Culti- var	Depth- Pond	Magn- Shape	Outlier- gram	n_meth- ods	Alco- hol	Fla- vanoids	ColorIn- tensity	Pro- line
70	Grigno- lino				2	12.2	1.3	2.9	718
96	Grigno- lino				2	12.5	2.3	2.6	937
60	Grigno- lino				1	12.4	0.6	2.0	520
74	Grigno- lino				1	13.0	2.9	3.4	985
79	Grigno- lino				1	12.3	1.9	3.4	750
97	Grigno- lino				1	11.8	1.0	2.5	625
111	Grigno- lino				1	11.5	2.6	2.9	562

	Culti- Wine var	Depth- Pond	Magn- Shape	Outlier- gram	n_meth- ods	Alco- hol	Fla- vanoids	ColorIn- tensity	Pro- line
122	Grigno- lino				1	11.6	5.1	6.0	465
159	Bar- bera				1	14.3	1.3	13.0	660

Deep Dive: The Grignolino Problem

Before we reach for generic conclusions, let's look at the data. Eight of nine outliers are Grignolino — zero from Barolo, one from Barbera. That asymmetry is itself an important finding. Let's understand what's driving it.

```
fda_all_idx <- unique(c(out_pond$outliers, og$outliers, ms$outliers))
cat("Outliers by cultivar:\n")
```

Outliers by cultivar:

```
for (cv in levels(cultivar)) {
  cv_n <- sum(cultivar == cv)
  cv_out <- sum(cultivar[fda_all_idx] == cv)
  cat(sprintf(" %s: %d outliers out of %d wines (%.0f%%)\n",
              cv, cv_out, cv_n, 100 * cv_out / cv_n))
}
```

```
Barolo: 0 outliers out of 59 wines (0%)
Grignolino: 8 outliers out of 71 wines (11%)
Barbera: 1 outliers out of 48 wines (2%)
```

Grignolino has by far the most variable chemical profile. Why? Let's examine what the flagged wines have in common. For each outlier, we compute its deviation from its own cultivar mean (in within-cultivar standard deviations) — this tells us how unusual a wine is *relative to its peer group*, not just globally:

```
# Compute within-cultivar z-scores for each outlier
outlier_profiles <- lapply(sort(fda_all_idx), function(i) {
  cv <- cultivar[i]
  cv_idx <- which(cultivar == cv)
```



```

cv_mean <- colMeans(wine[cv_idx, -1])
cv_sd <- apply(wine[cv_idx, -1], 2, sd)
dev <- (as.numeric(wine[i, -1]) - cv_mean) / cv_sd
names(dev) <- chem_names

# Flag chemicals deviating > 1.5 within-cultivar SDs
extreme <- which(abs(dev) > 1.5)
data.frame(
  Wine = i,
  Cultivar = as.character(cv),
  `Extreme Chemicals (> 1.5 cultivar SD)` =
    if (length(extreme) > 0)
      paste(sprintf("%s (%+.1f)", chem_names[extreme], dev[extreme]),
            collapse = ", ")
    else "none",
  n_extreme = length(extreme),
  check.names = FALSE
)
})

kable(bind_rows(outlier_profiles),
      caption = "Each outlier's unusual chemicals, measured against its own cultivar")

```

Table 4: Each outlier's unusual chemicals, measured against its own cultivar

	Culti- Winevar	Extreme Chemicals (> 1.5 cultivar SD)	n_ex- treme
60	Grig- no- lino	Ash (-2.8), Alkalinity (-2.9), Flavanoids (-2.1), Proanthocyanins (-2.0), OD280_OD315 (-1.9)	5
70	Grig- no- lino	Ash (-1.6), Magnesium (+3.4), NonflavPhenols (-1.8)	3
74	Grig- no- lino	Alkalinity (+2.9), Magnesium (+2.7), Phenols (+1.9), Proline (+3.0)	4
79	Grig- no- lino	Alkalinity (-1.6), Magnesium (+2.5), Proanthocyanins (+1.9)	3

	Culti- Winevar	Extreme Chemicals (> 1.5 cultivar SD)	n_ex- treme
96	Grig- no- lino	Magnesium (+4.0), Proanthocyanins (+2.7), Proline (+2.7)	3
97	Grig- no- lino	Ash (+1.6), Magnesium (+2.4), Flavanoids (-1.5), NonflavPhenols (-1.8)	4
111	Grig- no- lino	Alcohol (-1.5), MalicAcid (+1.8), Phenols (+1.7), Proanthocyanins (+3.2), Hue (-1.5)	5
122	Grig- no- lino	Ash (+3.1), Alkalinity (+2.5), Phenols (+1.7), Flavanoids (+4.2), ColorIntensity (+3.1), OD280_OD315 (+1.8)	6
159	Bar- bera	Alcohol (+2.2), MalicAcid (-1.5), Alkalinity (+1.6), Phenols (+3.1), Flavanoids (+1.8), Proanthocyanins (+3.8), ColorIntensity (+2.4)	7

A pattern jumps out: **Magnesium** appears in almost every Grignolino outlier. These wines have Magnesium levels 2–4 standard deviations above the Grignolino mean. Typical Grignolino has 94.5 mg/L; these outliers range from 107 to 162 mg/L. Other recurring deviations include high Proanthocyanins and unusual Alkalinity.

This is exactly the kind of systemic insight that a per-variable range check would bury. Yes, a spreadsheet filter could flag “Magnesium > 130,” but it wouldn’t reveal that these high-Magnesium wines *also* tend to have unusual Proanthocyanins, or that they cluster together in the Fourier domain. The curve representation lets us see the *compound pattern*.

Case Study: Wine 122 — Mislabeled or Just Unusual?

Wine 122 is the most extreme outlier: it ranks dead last in depth among all 71 Grignolinos (rank 71/71). Let’s look at where it sits relative to all three cultivar groups:

```
# Distance from Wine 122 to each cultivar median
case_wine <- 122

dist_to_median <- sapply(levels(cultivar), function(cv) {
  cv_idx <- which(cultivar == cv)
  fd_cv <- fd_wine[cv_idx]
  depths_cv <- depth.MBD(fd_cv)
  median_global <- cv_idx[which.max(depths_cv)]
  sqrt(sum((fd_wine$data[case_wine, ] - fd_wine$data[median_global, ])^2) *

```

```

    diff(fd_wine$argvals[1:2]))
})

cat(sprintf("Wine %d (%s) - distance to each cultivar median:\n", case_wine,
           cultivar[case_wine]))

```

Wine 122 (Grignolino) - distance to each cultivar median:

```

for (cv in names(dist_to_median)) {
  marker <- ifelse(cv == as.character(cultivar[case_wine]), " <-- own cultivar", "")
  cat(sprintf("  %s: %.2f%s\n", cv, dist_to_median[cv], marker))
}

```

```

Barolo: 10.67
Grignolino: 12.65 <-- own cultivar
Barbera: 13.24

```

```

closest <- names(which.min(dist_to_median))
cat(sprintf("\nClosest to: %s\n", closest))

```

Closest to: Barolo

```

# Highlight Wine 122 against its claimed cultivar and the cultivar it resembles
is_case <- rep(FALSE, n)
is_case[case_wine] <- TRUE

ggplot() +
  # Background: all curves, faded
  geom_line(data = filter(df_curves, Cultivar %in% c("Barolo", "Grignolino") &
                        !rep(is_case, each = m)),
            aes(x = t, y = value, group = curve, color = Cultivar),
            alpha = 0.15, linewidth = 0.3) +
  # Case wine: bold black
  geom_line(data = filter(df_curves, rep(is_case, each = m)),
            aes(x = t, y = value, group = curve),
            color = "black", linewidth = 1.2) +
  scale_color_manual(values = c("Barolo" = "#8B0000", "Grignolino" = "#DAA520")) +
  labs(title = sprintf("Wine %d: Labeled Grignolino, Looks Like Barolo", case_wine),
       x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

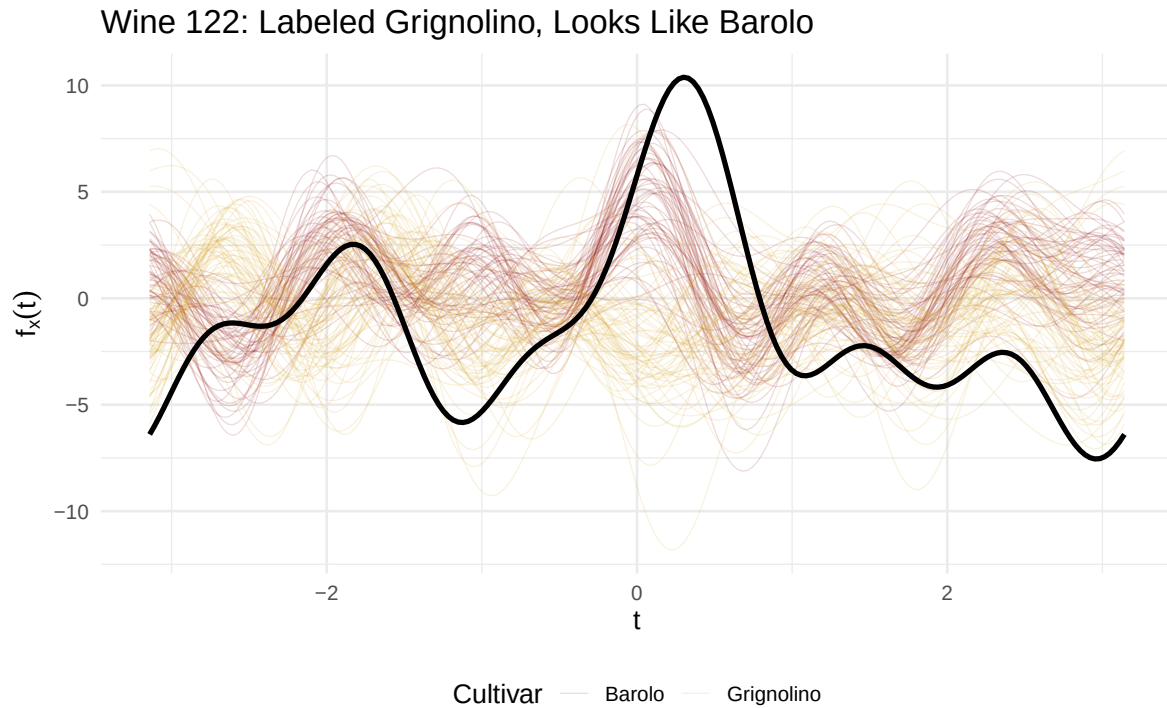


Figure 5: Wine 122 (black) overlaid on Barolo (red) and Grignolino (gold) curves — its profile sits squarely in Barolo territory.

```
# Show its chemical profile vs both cultivar means
cv_means <- sapply(c("Grignolino", "Barolo"), function(cv) {
  colMeans(wine[cultivar == cv, -1])
})

case_vals <- as.numeric(wine[case_wine, -1])

compare_122 <- data.frame(
  Chemical = chem_names,
  `Wine 122` = round(case_vals, 1),
  `Grignolino Mean` = round(cv_means[, "Grignolino"], 1),
  `Barolo Mean` = round(cv_means[, "Barolo"], 1),
  `Closer To` = ifelse(
    abs(case_vals - cv_means[, "Grignolino"]) < abs(case_vals - cv_means[, "Barolo"]),
    "Grignolino", "Barolo"),
  check.names = FALSE
)
```

```
kable(compare_122, caption = "Wine 122 vs cultivar means - closer to Barolo on most chemicals")
```

Table 5: Wine 122 vs cultivar means — closer to Barolo on most chemicals

	Chemical	Wine 122	Grignolino Mean	Barolo Mean	Closer To
Alcohol	Alcohol	11.6	12.3	13.7	Grignolino
MalicAcid	MalicAcid	2.0	1.9	2.0	Barolo
Ash	Ash	3.2	2.2	2.5	Barolo
Alkalinity	Alkalinity	28.5	20.2	17.0	Grignolino
Magnesium	Magnesium	119.0	94.5	106.3	Barolo
Phenols	Phenols	3.2	2.3	2.8	Barolo
Flavanoids	Flavanoids	5.1	2.1	3.0	Barolo
NonflavPhenols	NonflavPhenols	0.5	0.4	0.3	Grignolino
Proantho- cyanins	Proantho- cyanins	1.9	1.6	1.9	Barolo
ColorIntensity	ColorIntensity	6.0	3.1	5.5	Barolo
Hue	Hue	0.9	1.1	1.1	Grignolino
OD280_OD315	OD280_OD315	3.7	2.8	3.2	Barolo
Proline	Proline	465.0	519.5	1115.7	Grignolino

Wine 122 is closer to the Barolo median than to its own Grignolino median. On 13 chemicals, it resembles Barolo on the majority. Its Flavanoids (5.1) are $3\times$ the Grignolino average (2.1) but right at the Barolo average (2.98). Its Ash (3.2) and Alkalinity (28.5) are also Barolo-typical.

Quality controller’s action: this wine warrants a label audit. Either it was mislabeled during data collection, it comes from a vineyard on the Barolo-Grignolino boundary, or it was blended. The FDA analysis provides the evidence: it’s not just “unusual” — it’s unusual *in the direction of a specific other cultivar*. That’s a much more actionable finding than “Mahalanobis distance = 58.7.”

Case Study: Wines 70 and 96 — Shape Outliers Under the Microscope

These are the only wines flagged by *both* depth-ponderation and the outliergram. The outliergram specifically flags **shape anomalies** — wines whose curve crosses the population envelope in unexpected ways, even if their average level is unremarkable.

```
shape_cases <- c(70, 96)

# Their key chemistry
```

```

for (i in shape_cases) {
  cat(sprintf("\n--- Wine %d (%s) ---\n", i, cultivar[i]))
  cat(sprintf("  Magnesium:          %.0f mg/L  (Grignolino mean: 94.5, z = %+.1f)\n",
    wine[i, "Magnesium"], X[i, "Magnesium"]))
  cat(sprintf("  Proanthocyanins:  %.1f          (Grignolino mean: 1.6,  z = %+.1f)\n",
    wine[i, "Proanthocyanins"], X[i, "Proanthocyanins"]))
  cat(sprintf("  Ash:              %.1f g/L   (Grignolino mean: 2.2,  z = %+.1f)\n",
    wine[i, "Ash"], X[i, "Ash"]))

  # Fuzzy membership
  fcm_all <- cluster.fcm(fd_wine, ncl = 3, seed = 123)
  mem <- round(fcm_all$membership[i, ], 3)
  cat(sprintf("  Fuzzy membership: [%.3f, %.3f, %.3f] - max = %.3f\n",
    mem[1], mem[2], mem[3], max(mem)))
}

```

```

--- Wine 70 (Grignolino) ---
Magnesium:          151 mg/L  (Grignolino mean: 94.5, z = +3.6)
Proanthocyanins:    2.5        (Grignolino mean: 1.6,  z = +1.6)
Ash:                1.8 g/L   (Grignolino mean: 2.2,  z = -2.2)
Fuzzy membership: [0.149, 0.445, 0.406] - max = 0.445

```

```

--- Wine 96 (Grignolino) ---
Magnesium:          162 mg/L  (Grignolino mean: 94.5, z = +4.4)
Proanthocyanins:    3.3        (Grignolino mean: 1.6,  z = +3.0)
Ash:                2.2 g/L   (Grignolino mean: 2.2,  z = -0.6)
Fuzzy membership: [0.143, 0.312, 0.545] - max = 0.545

```

```

is_shape_case <- rep(FALSE, n)
is_shape_case[shape_cases] <- TRUE

ggplot() +
  geom_line(data = filter(df_curves, !rep(is_shape_case, each = m)),
    aes(x = t, y = value, group = curve, color = Cultivar),
    alpha = 0.12, linewidth = 0.3) +
  geom_line(data = filter(df_curves, rep(is_shape_case, each = m)),
    aes(x = t, y = value, group = curve),
    color = "black", linewidth = 1.1) +
  scale_color_manual(values = c("Barolo" = "#8B0000", "Grignolino" = "#DAA520",
    "Barbera" = "#2E8B57")) +

```

```
labs(title = "Shape Outliers: Wines 70 and 96",
     subtitle = "Flagged by both depth-ponderation and outliergram",
     x = expression(t), y = expression(f[x](t))) +
theme(legend.position = "bottom")
```

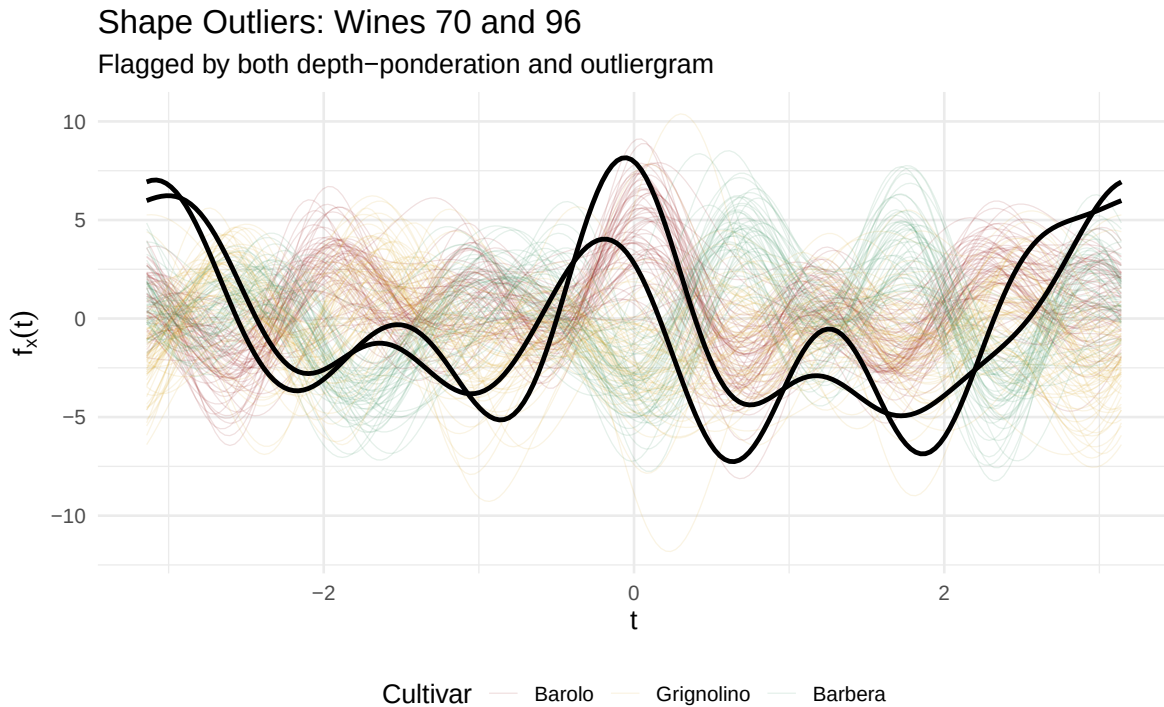


Figure 6: Wines 70 and 96 (black) weave through the Grignolino bundle in an atypical pattern — their shapes cross other cultivars' territory.

Both wines have extremely high Magnesium (151 and 162 mg/L vs a Grignolino average of 94.5). But they also deviate on *different* secondary chemicals — Wine 70 has very low Ash and NonflavPhenols, while Wine 96 has extreme Proanthocyanins (3.3, vs mean 1.6). What makes them *shape* outliers rather than mere magnitude outliers is that this combination of high and low values creates curves that *cross* the normal Grignolino band, dipping below it in some Fourier regions and rising above it in others.

Quality controller's action: investigate the vineyard. Extreme Magnesium in an otherwise Grignolino-like wine suggests a **soil chemistry anomaly** — Magnesium uptake in grapes is strongly influenced by soil composition and rootstock. These wines may come from a plot with unusual mineral content. The shape anomaly means a simple recalibration won't fix the issue; the underlying growing conditions need review.

Case Study: Wine 159 — The Lone Barbera Outlier

Wine 159 is the only Barbera flagged, and it's an outlier for very different reasons than the Grignolinos:

```
i <- 159
cat(sprintf("Wine %d (%s)\n", i, cultivar[i]))
```

Wine 159 (Barbera)

```
cat(sprintf("  Color Intensity: %.1f  (Barbera mean: 7.4, population z = %+.1f)\n",
            wine[i, "ColorIntensity"], X[i, "ColorIntensity"]))
```

Color Intensity: 13.0 (Barbera mean: 7.4, population z = +3.4)

```
cat(sprintf("  Alcohol:          %.1f%%  (Barbera mean: 13.2%%, z = %+.1f)\n",
            wine[i, "Alcohol"], X[i, "Alcohol"]))
```

Alcohol: 14.3% (Barbera mean: 13.2%, z = +1.6)

```
cat(sprintf("  Phenols:          %.1f  (Barbera mean: 1.7, z = %+.1f)\n",
            wine[i, "Phenols"], X[i, "Phenols"]))
```

Phenols: 2.8 (Barbera mean: 1.7, z = +0.8)

```
cat(sprintf("  Proanthocyanins: %.1f  (Barbera mean: 1.2, z = %+.1f)\n",
            wine[i, "Proanthocyanins"], X[i, "Proanthocyanins"]))
```

Proanthocyanins: 2.7 (Barbera mean: 1.2, z = +1.9)

```
# Within-cultivar rank
cv_idx <- which(cultivar == "Barbera")
fd_cv <- fd_wine[cv_idx]
depths_cv <- depth.MBD(fd_cv)
pos_in_cv <- which(cv_idx == i)
rank_in_cv <- rank(-depths_cv)[pos_in_cv]
cat(sprintf("\n  Depth rank within Barbera: %d / %d (dead last)\n",
            rank_in_cv, length(cv_idx)))
```


Depth rank within Barbera: 48 / 48 (dead last)

```
is_159 <- rep(FALSE, n)
is_159[159] <- TRUE

ggplot() +
  geom_line(data = filter(df_curves, Cultivar == "Barbera" & !rep(is_159, each = m)),
    aes(x = t, y = value, group = curve),
    color = "#2E8B57", alpha = 0.3, linewidth = 0.4) +
  geom_line(data = filter(df_curves, rep(is_159, each = m)),
    aes(x = t, y = value, group = curve),
    color = "black", linewidth = 1.2) +
  labs(title = "Wine 159: The Barbera Outlier",
    subtitle = "Extreme Color Intensity (z = +3.4) pushes it to the edge of the Barbera envelope",
    x = expression(t), y = expression(f[x](t)))
```

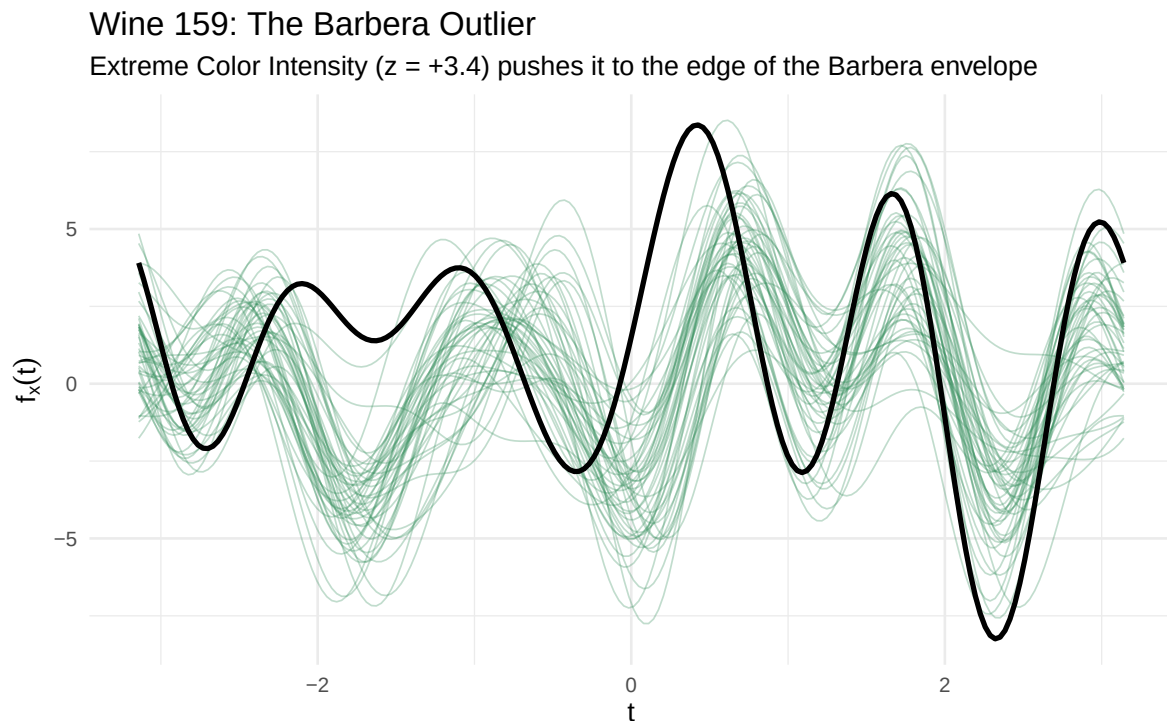


Figure 7: Wine 159 (black) against the Barbera group — high amplitude, but still within Barbera's general shape. A magnitude outlier, not a shape outlier.

Wine 159 has the highest Color Intensity in the entire dataset (13.0 vs a Barbera mean of 7.4).

Its Phenols and Proanthocyanins are also well above the Barbera average. But the curve plot reveals something important: **it follows the Barbera shape**. The curve runs parallel to its cultivar mates, just with higher amplitude. This is a *magnitude* outlier — the overall chemical levels are elevated — not a *shape* outlier.

Quality controller's action: re-test the Color Intensity measurement. A reading of 13.0 is almost double the cultivar mean, which could indicate either a genuinely concentrated wine (late harvest, low yield) or an analytical error. The Phenol and Proanthocyanin values corroborate the “concentrated” hypothesis — they’re consistently elevated, not randomly scattered. If re-testing confirms the values, this wine is likely a natural outlier from an unusually productive or concentrated lot, not a quality defect.

Summary: The Outlier Action Matrix

Putting it all together, here’s what the quality controller walks away with — not “these wines are unusual” (a spreadsheet could tell you that), but *what kind* of unusual, *why*, and *what to do about it*:

```
actions <- data.frame(
  Wine = c(122, "70, 96", 159, "60, 74, 79, 97, 111"),
  Type = c("Cultivar mismatch", "Shape outlier (soil/mineral)",
           "Magnitude outlier (concentration)", "Peripheral Grignolino"),
  `Key Finding` = c(
    "Closer to Barolo median than Grignolino; 6 chemicals match Barolo profile",
    "Extreme Magnesium (151-162 mg/L); curves cross the population envelope",
    "Color Intensity 2× cultivar mean; all values elevated proportionally",
    "Various extreme chemicals (Mg, Ash, Alkalinity); fringes of Grignolino cluster"
  ),
  Action = c(
    "Label audit - verify cultivar assignment; check vineyard of origin",
    "Investigate soil chemistry and rootstock at source vineyard",
    "Re-test Color Intensity; if confirmed, flag as concentrated lot",
    "Review against cultivar specification; note in batch records"
  ),
  check.names = FALSE
)
kable(actions, caption = "Outlier triage: from flag to action")
```

Table 6: Outlier triage: from flag to action

Wine	Type	Key Finding	Action
122	Cultivar mismatch	Closer to Barolo median than Grignolino; 6 chemicals match Barolo profile	Label audit — verify cultivar assignment; check vineyard of origin
70, 96	Shape outlier (soil/mineral)	Extreme Magnesium (151–162 mg/L); curves cross the population envelope	Investigate soil chemistry and rootstock at source vineyard
159	Magnitude outlier (concentration)	Color Intensity $2\times$ cultivar mean; all values elevated proportionally	Re-test Color Intensity; if confirmed, flag as concentrated lot
60, 74, 79, 97, 111	Peripheral Grignolino	Various extreme chemicals (Mg, Ash, Alkalinity); fringes of Grignolino cluster	Review against cultivar specification; note in batch records

This is the payoff of the functional approach. A classical Mahalanobis analysis would have flagged most of these same wines — but it would have given the quality controller a list of numbers with no indication of *why* each wine is unusual or *what to do about it*. The FDA pipeline delivers typed outliers (magnitude vs shape vs cultivar mismatch), visual diagnostics (the curve overlays above), and domain-specific action items. That’s the difference between a statistical report and a quality decision.

Comparison: What Would Standard Statistics Give You?

A classically trained analyst might reach for **Mahalanobis distance** — the standard multivariate outlier detector. It measures how far each wine is from the population mean, accounting for correlations between chemicals. Wines exceeding a χ^2 critical value are flagged.

```
# Classical Mahalanobis distance outlier detection
mahal <- mahalanobis(X, colMeans(X), cov(X))
cutoff_chi <- qchisq(0.975, df = ncol(X))
mahal_outliers <- which(mahal > cutoff_chi)

cat(sprintf("Mahalanobis outliers (chi-sq cutoff = %.1f, df = %d):\n",
            cutoff_chi, ncol(X)))
```

```
Mahalanobis outliers (chi-sq cutoff = 24.7, df = 13):
```

```
cat("  Flagged:", sort(mahal_outliers), "\n")
```

Flagged: 14 60 70 72 74 96 97 111 116 122 159 160

```
cat("  Count:", length(mahal_outliers), "\n")
```

Count: 12

```
# Compare with FDA results
fda_all <- unique(c(out_pond$outliers, og$outliers, ms$outliers))
overlap <- intersect(mahal_outliers, fda_all)
fda_only <- setdiff(fda_all, mahal_outliers)
mahal_only <- setdiff(mahal_outliers, fda_all)

cat("Overlap between methods:\n")
```

Overlap between methods:

```
cat("  Both flag:          ", sort(overlap), "\n")
```

Both flag: 60 70 74 96 97 111 122 159

```
cat("  FDA only:          ", sort(fda_only), "\n")
```

FDA only: 79

```
cat("  Mahalanobis only:    ", sort(mahal_only), "\n")
```

Mahalanobis only: 14 72 116 160

```
# Show Mahalanobis distances for FDA-flagged wines
compare_df <- data.frame(
  Wine = sort(union(mahal_outliers, fda_all)),
  Cultivar = as.character(cultivar[sort(union(mahal_outliers, fda_all))])
)
compare_df$Mahal_Dist <- round(mahal[compare_df$Wine], 1)
compare_df$Mahal_Flag <- compare_df$Wine %in% mahal_outliers
```

```

compare_df$FDA_Flag <- compare_df$Wine %in% fda_all
compare_df$FDA_Methods <- sapply(compare_df$Wine, function(w) {
  methods <- c()
  if (w %in% out_pond$outliers) methods <- c(methods, "Depth")
  if (w %in% og$outliers) methods <- c(methods, "Outliergram")
  if (w %in% ms$outliers) methods <- c(methods, "MS-plot")
  paste(methods, collapse = " + ")
})

kable(compare_df, caption = "Mahalanobis vs FDA: flagged wines compared")

```

Table 7: Mahalanobis vs FDA: flagged wines compared

Wine	Cultivar	Mahal_Dist	Mahal_Flag	FDA_Flag	FDA_Methods
14	Barolo	25.0	TRUE	FALSE	
60	Grignolino	27.6	TRUE	TRUE	Depth
70	Grignolino	38.7	TRUE	TRUE	Depth + Outliergram
72	Grignolino	26.5	TRUE	FALSE	
74	Grignolino	38.0	TRUE	TRUE	Depth
79	Grignolino	24.2	FALSE	TRUE	Depth
96	Grignolino	38.2	TRUE	TRUE	Depth + Outliergram
97	Grignolino	25.2	TRUE	TRUE	Depth
111	Grignolino	35.2	TRUE	TRUE	Depth
116	Grignolino	26.9	TRUE	FALSE	
122	Grignolino	58.7	TRUE	TRUE	Depth
159	Barbera	36.5	TRUE	TRUE	Depth
160	Barbera	26.1	TRUE	FALSE	

The overlap is substantial — **as the distance preservation theorem predicts**, both approaches are working with the same underlying Euclidean geometry. But notice the difference in what you *get*:

	Mahalanobis Distance	FDA Outlier Methods
Output	One number per wine	Three diagnostic plots + typed flags
Tells you <i>that</i>	Yes	Yes
Tells you <i>how</i>	No — just “far from center”	Yes — magnitude vs shape vs rank structure
Visual diagnostic	None (abstract distance)	Overlay outlier curves on the population

	Mahalanobis Distance	FDA Outlier Methods
Assumes	Multivariate normality	No distributional assumption
Multiple aspects	Single axis of outlyingness	Three complementary axes

Mahalanobis distance is a fine screening tool, and in this dataset it largely agrees with the FDA methods. But when it flags Wine 122 with a distance of 58.7, it can't tell you *why* — is every chemical extreme, or is the pattern unusual? The FDA approach answers that: depth-ponderation says the wine is far from the population center, the outliergram says the shape is unusual, and the magnitude-shape plot says the chemical *ratios* aren't distorted. For a quality controller, that distinction means the difference between re-running the test (magnitude issue) and investigating the vineyard (shape issue).

Clustering: Curves Know Their Groups

Can the curves recover the cultivar labels *without ever seeing them*? This is the acid test for cultivar authenticity verification.

Prepare Clean Data

First, we remove wines flagged by two or more outlier methods — they could distort cluster centers.

```
multi_flagged <- outlier_flags$Wine[outlier_flags$n_methods >= 2]
cat("Removing", length(multi_flagged), "multi-flagged outliers\n")
```

Removing 2 multi-flagged outliers

```
if (length(multi_flagged) > 0) {
  keep <- setdiff(1:n, multi_flagged)
  fd_clean <- fd_wine[keep]
  cultivar_clean <- cultivar[keep]
  X_clean <- X[keep, ]
} else {
  fd_clean <- fd_wine
  cultivar_clean <- cultivar
  X_clean <- X
}

cat("Clean dataset:", nrow(fd_clean$data), "wines\n")
```

Clean dataset: 176 wines

Functional K-Means

```
km_fda <- cluster.kmeans(fd_clean, ncl = 3, nstart = 20, seed = 123)

# Confusion matrix with label permutation for optimal matching
conf <- table(Cluster = km_fda$cluster, True = cultivar_clean)

# Try all 6 permutations of labels
best_acc <- 0
best_perm <- NULL
perms <- list(c(1,2,3), c(1,3,2), c(2,1,3), c(2,3,1), c(3,1,2), c(3,2,1))
for (perm in perms) {
  acc <- sum(km_fda$cluster == perm[as.numeric(cultivar_clean)])
  if (acc > best_acc) {
    best_acc <- acc
    best_perm <- perm
  }
}

cat("Confusion matrix:\n")
```

Confusion matrix:

```
print(conf)
```

	True		
Cluster	Barolo	Grignolino	Barbera
1	59	2	0
2	0	64	0
3	0	3	48

```
cat(sprintf("\nBest matching accuracy: %d / %d = %.1f%%\n",
            best_acc, length(cultivar_clean),
            100 * best_acc / length(cultivar_clean)))
```

Best matching accuracy: 171 / 176 = 97.2%

Comparison: Standard K-Means on Raw Data

The distance preservation theorem predicts that standard k-means on the original 13 variables should give the same clustering accuracy. Let's verify — and then examine what each approach gives you *beyond* the cluster assignments.

```
set.seed(123)
km_raw <- kmeans(X_clean, centers = 3, nstart = 20)

best_acc_raw <- 0
best_perm_raw <- NULL
for (perm in perms) {
  acc <- sum(km_raw$cluster == perm[as.numeric(cultivar_clean)])
  if (acc > best_acc_raw) {
    best_acc_raw <- acc
    best_perm_raw <- perm
  }
}

cat(sprintf("FDA k-means accuracy:  %.1f%%\n",
            100 * best_acc / length(cultivar_clean)))
```

FDA k-means accuracy: 97.2%

```
cat(sprintf("Raw k-means accuracy:  %.1f%%\n",
            100 * best_acc_raw / length(cultivar_clean)))
```

Raw k-means accuracy: 97.2%

Same accuracy — as guaranteed by the distance preservation theorem. So why bother with Andrews curves at all? The difference is in what you can **do with the result** after clustering.

Standard k-means gives you 13-number centroids. Each cluster center is a vector of 13 chemical values. To present this to a non-technical audience, you need a table or a parallel coordinate plot — neither of which makes it easy to “see” the differences between cultivars at a glance:

```
# Standard k-means centers: a table of numbers
raw_center_df <- as.data.frame(round(km_raw$centers, 2))
raw_center_df$Cluster <- paste("Cluster", 1:3)
kable(raw_center_df, caption = "Standard k-means: 13 numbers per cluster center")
```


Table 9: Standard k-means: 13 numbers per cluster center

Al- co- hol	Mal- i- cAcid	Ash	Al- ka- lin- ity	Mag- ne- sium	Phe- nols	Fla- vanoids	Non- flavPhe- nols	Proan- tho- cyanins	Color- Inten- sity	Hue	OD280 _{nm}	Pro- line	Clus- ter
-	-	-	0.19	-	-	0.03	-0.01	0.03	-0.90	0.45	0.26	-	Clus- ter 1
0.92	0.38	0.47		0.55	0.07							0.76	
0.86	-	0.38	-	0.51	0.89	0.99	-0.56	0.54	0.19	0.47	0.79	1.13	Clus- ter 2
	0.30		0.62										
0.16	0.87	0.19	0.52	-	-	-	0.72	-0.78	0.94	-	-1.29	-	Clus- ter 3
				0.08	0.98	1.21				1.16		0.41	

Functional k-means gives you center *curves*. Each cluster is represented by a curve that *encodes the same 13 chemicals*, but as a single visual object you can compare, overlay, and present:

```
n_clusters <- nrow(km_fda$centers$data)
df_centers <- data.frame(
  t = rep(t_grid, n_clusters),
  value = as.vector(t(km_fda$centers$data)),
  Cluster = factor(rep(1:n_clusters, each = m))
)

ggplot(df_centers, aes(x = t, y = value, color = Cluster)) +
  geom_line(linewidth = 1.3) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "Functional K-Means Cluster Centers",
       subtitle = "Each curve encodes the typical chemical fingerprint of a cultivar group",
       x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")
```

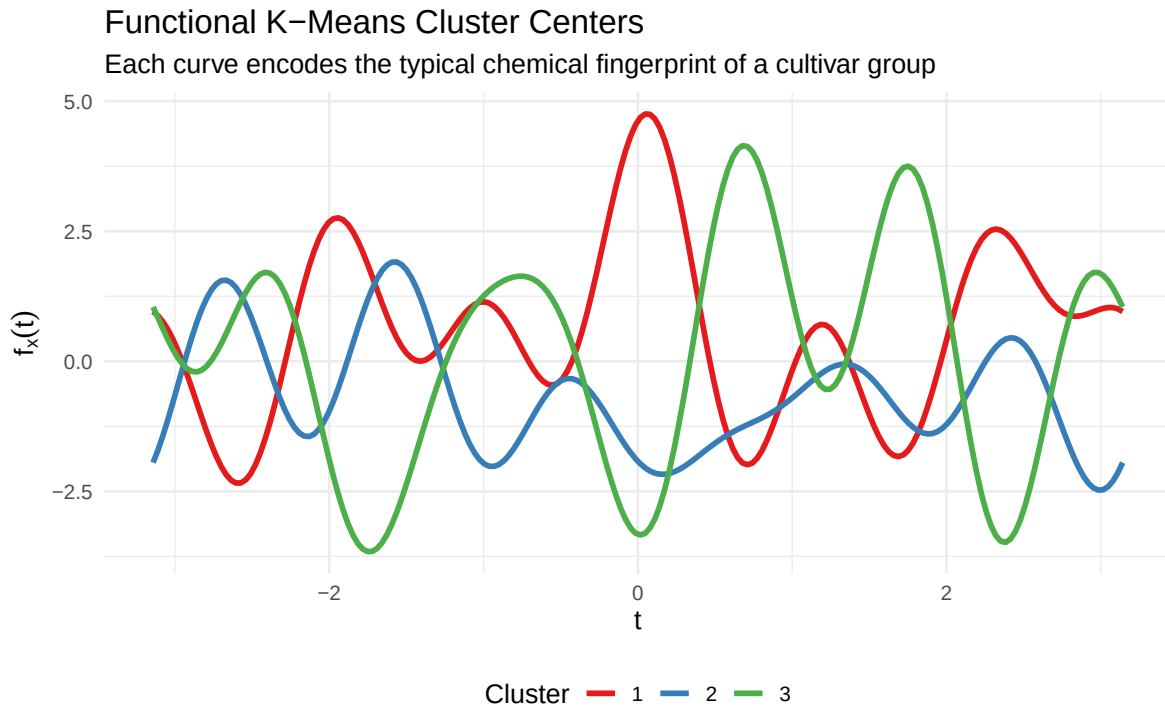


Figure 8: FDA cluster centers as curves — each curve encodes the 13-chemical centroid as a single visual fingerprint.

A winemaker looking at this plot immediately sees: cluster 1 and cluster 3 are mirror images (one high where the other is low), while cluster 2 sits in between. That’s the kind of gestalt understanding that a table of 39 numbers can’t deliver. When the certification board asks “how are your cultivars different?”, a single curve overlay answers the question — without requiring the audience to parse a table.

The cluster *assignments* are mathematically identical. The difference is in **communication**: center curves are self-explanatory; center vectors require explanation.

Fuzzy C-Means: Quantifying Uncertainty

Binary cluster assignments hide important nuance. A wine labeled “Grignolino” that clusters 55% Grignolino / 45% Barbera is a very different story from one that’s 99% Grignolino.

```
fcm <- cluster.fcm(fd_clean, ncl = 3, seed = 123)

# Identify boundary wines (max membership < 0.6)
```

```

max_mem <- apply(fcm$membership, 1, max)
boundary_idx <- which(max_mem < 0.6)

cat(length(boundary_idx), "boundary wines with max membership < 0.6\n")

```

25 boundary wines with max membership < 0.6

```

n_clean <- nrow(fd_clean$data)
is_boundary <- rep(FALSE, n_clean)
is_boundary[boundary_idx] <- TRUE

df_clean_curves <- data.frame(
  t = rep(t_grid, n_clean),
  value = as.vector(t(fd_clean$data)),
  curve = rep(1:n_clean, each = m),
  Cultivar = rep(cultivar_clean, each = m),
  boundary = rep(is_boundary, each = m)
)

ggplot() +
  geom_line(data = filter(df_clean_curves, !boundary),
    aes(x = t, y = value, group = curve, color = Cultivar),
    alpha = 0.2, linewidth = 0.3) +
  geom_line(data = filter(df_clean_curves, boundary),
    aes(x = t, y = value, group = curve),
    color = "black", linewidth = 0.8, alpha = 0.7) +
  scale_color_manual(values = c("Barolo" = "#8B0000",
    "Grignolino" = "#DAA520",
    "Barbera" = "#2E8B57")) +
  labs(title = "Boundary Wines Highlighted on Andrews Curves",
    subtitle = sprintf("%d wines with max cluster membership < 0.6", length(boundary_idx)),
    x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

Boundary Wines Highlighted on Andrews Curves

25 wines with max cluster membership < 0.6

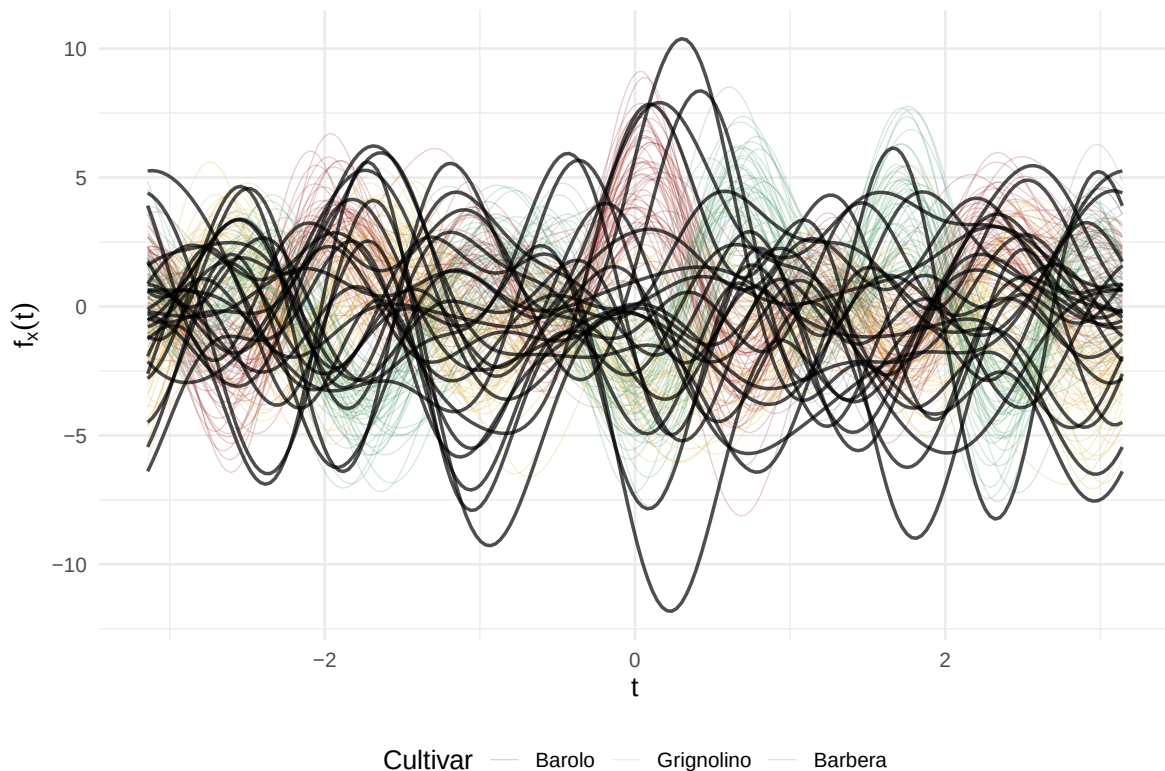


Figure 9: Boundary wines (bold) sit between cultivar clusters — candidates for blend investigation or appellation review.

For a certification body, this quantified uncertainty is far more actionable than a binary classification. Wines with split membership may come from a vineyard on the boundary between appellations, or represent a blend that wasn't fully declared.

Do Cultivar Means Really Differ?

Visual separation is suggestive, but for regulatory claims (“our Barolo is chemically distinct”), you need a p-value.

```
gt <- group.test(fd_wine, cultivar, n.perm = 1000, statistic = "ratio")
cat(sprintf("Permutation test (F-ratio statistic):\n"))
```

Permutation test (F-ratio statistic):

```
cat(sprintf("  Observed statistic: %.2f\n", gt$statistic))
```

Observed statistic: 0.78

```
cat(sprintf("  p-value:           %s\n",
            ifelse(gt$p.value == 0,
                  sprintf("< 1/%d = %.4f", gt$n.perm, 1/gt$n.perm),
                  sprintf("%.4f", gt$p.value))))
```

p-value: < 1/1000 = 0.0010

Bootstrap Confidence Intervals per Cultivar

The permutation test tells us the means differ; the bootstrap CIs show *where* on the chemical profile the cultivars diverge.

```
cultivar_levels <- levels(cultivar)
ci_list <- list()

for (cv in cultivar_levels) {
  idx <- which(cultivar == cv)
  fd_sub <- fd_wine[idx]

  ci <- fdata.bootstrap.ci(
    fd_sub,
    statistic = function(x) as.numeric(mean(x)$data),
    n.boot = 500,
    seed = 42
  )

  ci_list[[cv]] <- data.frame(
    t = t_grid,
    mean = ci$estimate,
    ci_lower = ci$ci.lower,
    ci_upper = ci$ci.upper,
    Cultivar = cv
  )
}

df_ci <- bind_rows(ci_list)
```

```
ggplot(df_ci, aes(x = t, fill = Cultivar, color = Cultivar)) +
  geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper), alpha = 0.2, color = NA) +
  geom_line(aes(y = mean), linewidth = 1) +
  scale_color_manual(values = c("Barolo" = "#8B0000",
                                "Grignolino" = "#DAA520",
                                "Barbera" = "#2E8B57")) +
  scale_fill_manual(values = c("Barolo" = "#8B0000",
                                "Grignolino" = "#DAA520",
                                "Barbera" = "#2E8B57")) +
  labs(title = "Mean Andrews Curves with 95% Bootstrap CIs",
       x = expression(t), y = expression(bar(f)(t))) +
  theme(legend.position = "bottom")
```

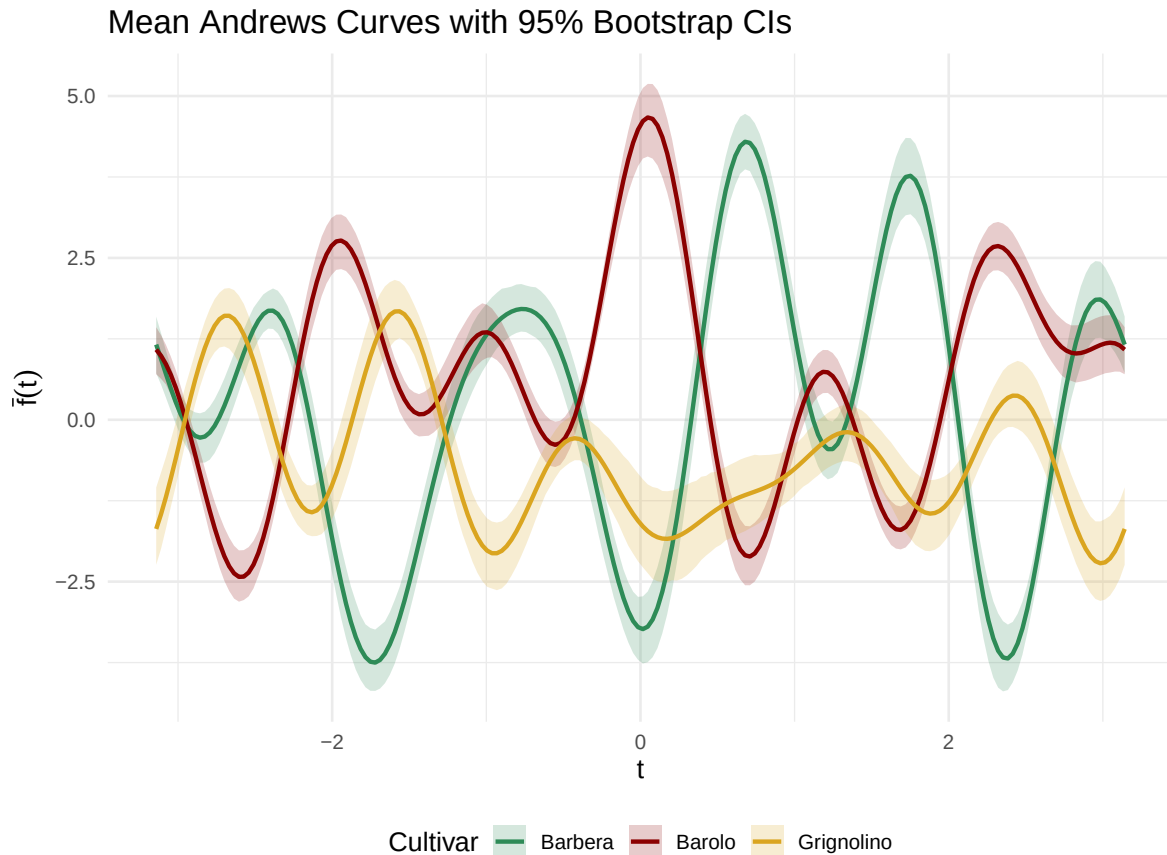


Figure 10: Mean curves with 95% bootstrap CIs — non-overlapping regions identify the Fourier frequencies (and corresponding chemicals) that discriminate cultivars.

Regions where the confidence ribbons don't overlap correspond to Fourier frequencies — and through the Andrews mapping, specific chemicals — that reliably distinguish cultivars. This feeds directly into denomination-of-origin specifications: which chemicals should be tested to verify a wine's claimed cultivar?

Which Chemicals Matter Most?

If your lab budget allows testing only 5 of the 13 chemicals, which 5 should you keep? Functional PCA on the Andrews curves answers this.

```
fpca <- fdata2pc(fd_wine, ncomp = 5)

# Variance explained
var_explained <- fpca$d^2 / sum(fpca$d^2) * 100
cat("Variance explained by first 5 PCs:\n")
```

Variance explained by first 5 PCs:

```
for (i in 1:5) {
  cat(sprintf("  PC%d: %.1f%%\n", i, var_explained[i]))
}
```

```
PC1: 45.0%
PC2: 24.1%
PC3: 13.9%
PC4:  8.8%
PC5:  8.2%
```

```
cat(sprintf("  Total (PC1-5): %.1f%%\n", sum(var_explained[1:5])))
```

```
Total (PC1-5): 100.0%
```

```
df_screes <- data.frame(
  PC = factor(1:5),
  Variance = var_explained[1:5],
  Cumulative = cumsum(var_explained[1:5])
)

p_bar <- ggplot(df_screes, aes(x = PC, y = Variance)) +
```

```

geom_col(fill = "steelblue", alpha = 0.8) +
labs(y = "% Variance Explained", x = "Principal Component") +
theme_minimal()

p_cum <- ggplot(df_screes, aes(x = as.numeric(PC), y = Cumulative)) +
  geom_line(linewidth = 1, color = "darkred") +
  geom_point(size = 3, color = "darkred") +
  geom_hline(yintercept = 90, linetype = "dashed", color = "gray50") +
  labs(y = "Cumulative %", x = "Principal Component") +
  scale_x_continuous(breaks = 1:5) +
  theme_minimal()

p_bar + p_cum + plot_annotation(title = "FPCA Variance Decomposition")

```

FPCA Variance Decomposition

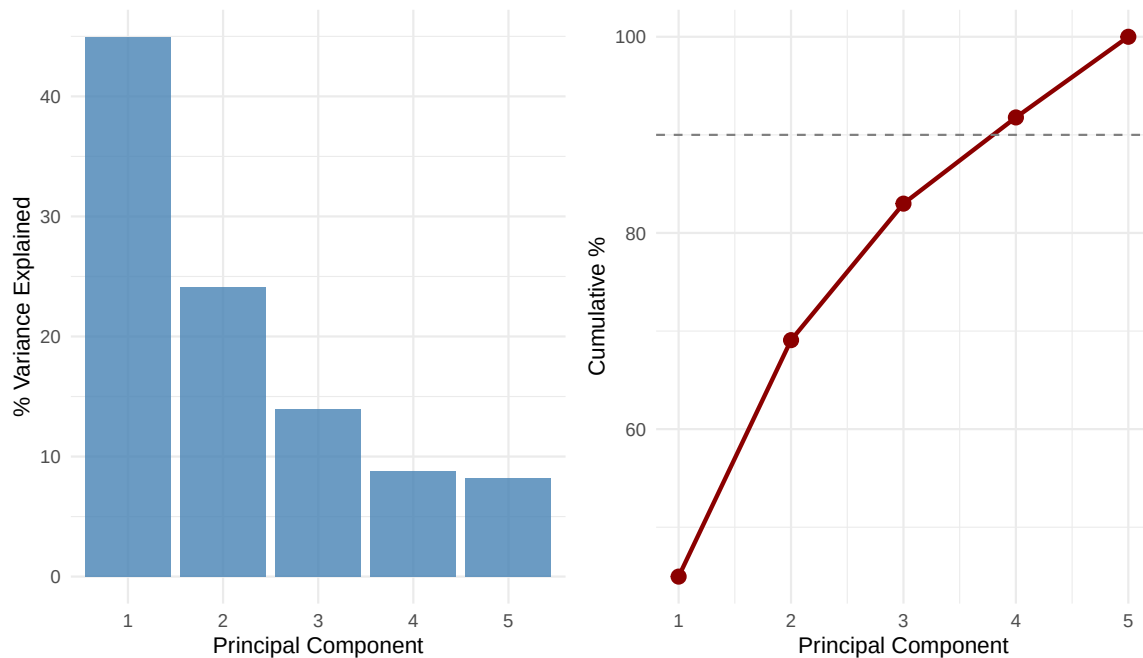


Figure 11: Scree plot — the first two PCs capture the bulk of variation.

```

df_scores <- data.frame(
  PC1 = fpca$x[, 1],
  PC2 = fpca$x[, 2],
  Cultivar = cultivar
)

```



```
)

ggplot(df_scores, aes(x = PC1, y = PC2, color = Cultivar)) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_manual(values = c("Barolo" = "#8B0000",
                                "Grignolino" = "#DAA520",
                                "Barbera" = "#2E8B57")) +
  labs(title = "FPCA Score Plot",
       x = sprintf("PC1 (%.1f%%)", var_explained[1]),
       y = sprintf("PC2 (%.1f%%)", var_explained[2])) +
  theme(legend.position = "bottom")
```

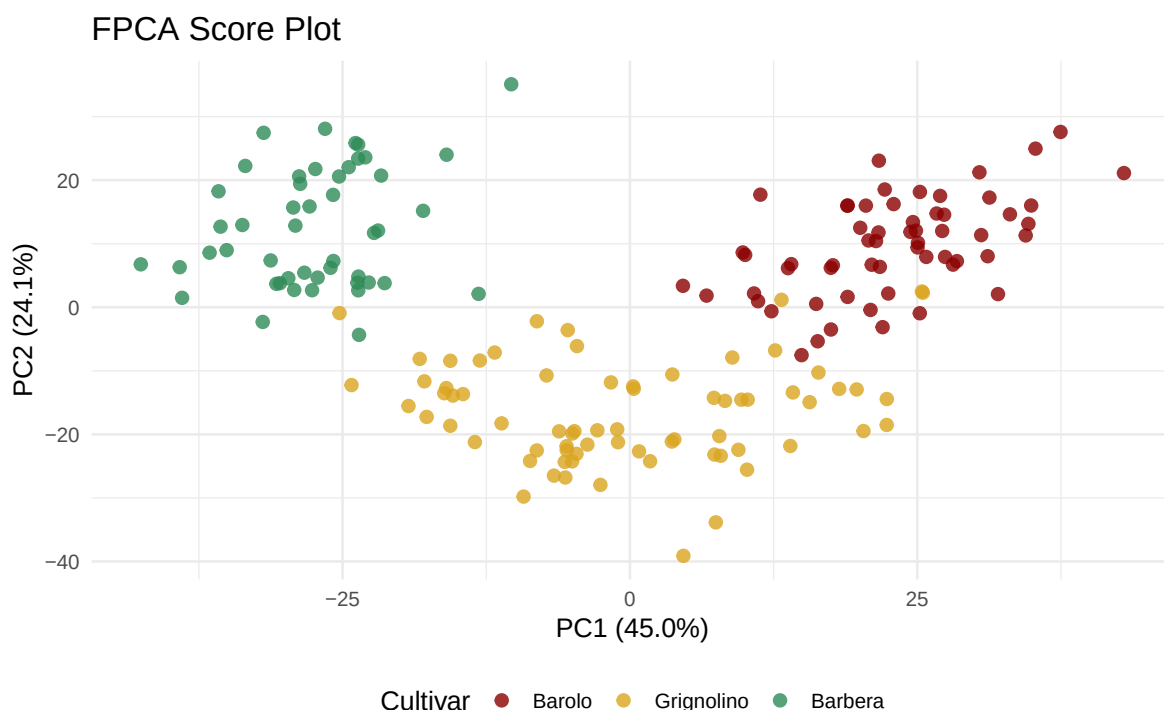


Figure 12: FPCA score plot — cultivars separate cleanly in the first two principal components.

Projecting Back to Original Variables

The FPCA eigenfunctions live in Fourier space. To identify which of the 13 original chemicals drive each PC, we project each eigenfunction back onto the Andrews basis vectors. The inner product of an eigenfunction with the j -th basis function gives the “loading” of variable j on that PC.

```

# Reconstruct the basis matrix used in andrews_transform
p <- ncol(X)
basis <- matrix(0, m, p)
basis[, 1] <- 1 / sqrt(2)
for (j in 2:p) {
  k <- ceiling((j - 1) / 2)
  if (j %% 2 == 0) {
    basis[, j] <- sin(k * t_grid)
  } else {
    basis[, j] <- cos(k * t_grid)
  }
}

# Project eigenfunctions onto basis: inner product (trapezoidal integration)
dt <- diff(t_grid[1:2])
chem_names <- colnames(wine)[-1]
loading_list <- list()

for (pc_idx in 1:3) {
  eigen_vals <- fpca$rotation$data[pc_idx, ]
  loadings <- as.numeric(t(basis) %*% eigen_vals) * dt
  loading_list[[pc_idx]] <- data.frame(
    Chemical = chem_names,
    Loading = loadings,
    AbsLoading = abs(loadings),
    PC = paste0("PC", pc_idx)
  )
}

df_loadings <- bind_rows(loading_list)

ggplot(df_loadings, aes(x = reorder(Chemical, AbsLoading), y = Loading,
                             fill = Loading > 0)) +
  geom_col(alpha = 0.8, show.legend = FALSE) +
  coord_flip() +
  facet_wrap(~ PC, scales = "free_x") +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "coral")) +
  labs(title = "Chemical Loadings on Principal Components",
       subtitle = "Larger bars = more important for discriminating cultivars",
       x = NULL, y = "Loading (inner product with eigenfunction)") +
  theme(strip.text = element_text(face = "bold"))

```

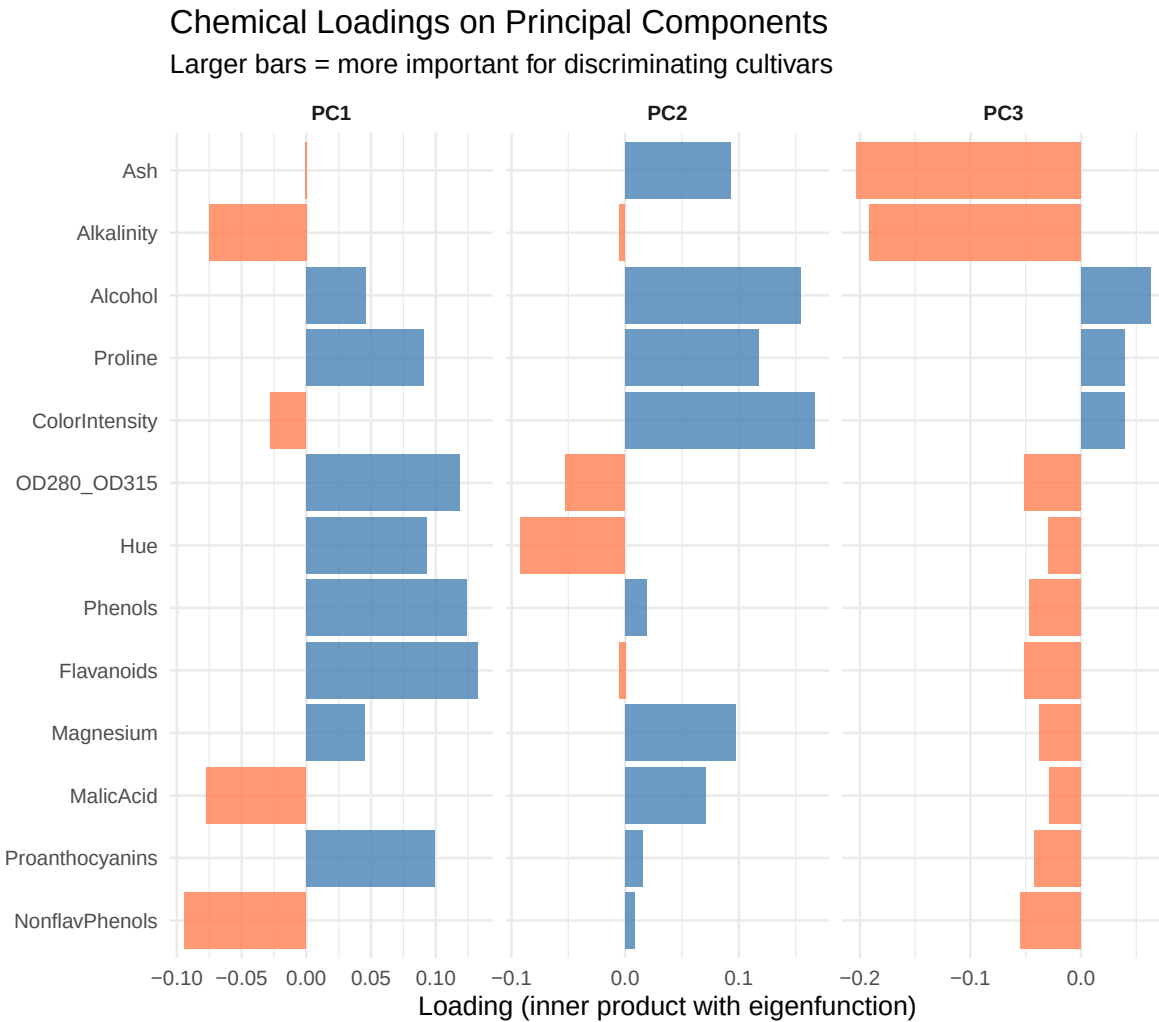


Figure 13: Variable importance per PC — the chemicals with largest loadings carry the most discriminating information.

The chemicals with the largest loadings on PC1 and PC2 carry the most discriminating information. For a cost-conscious lab, this is the difference between running 13 assays and running 5 without losing classification power.

Comparison: Standard PCA on Raw Data

A natural question: why not just run `prcomp()` on the 13×178 matrix? Let's do exactly that and compare.

```
pca_raw <- prcomp(X, center = FALSE) # already centered & scaled
var_raw <- pca_raw$sdev^2 / sum(pca_raw$sdev^2) * 100

cat("Variance explained - standard PCA vs FPCA:\n")
```

Variance explained - standard PCA vs FPCA:

```
cat(sprintf("  %-6s %8s %8s\n", "PC", "Std PCA", "FPCA"))
```

PC	Std PCA	FPCA
----	---------	------

```
for (i in 1:5) {
  cat(sprintf("  PC%-3d  %6.1f%%  %6.1f%%\n", i, var_raw[i], var_explained[i]))
}
```

PC1	36.2%	45.0%
PC2	19.2%	24.1%
PC3	11.1%	13.9%
PC4	7.1%	8.8%
PC5	6.6%	8.2%

```
# Scores are (nearly) identical
cat("Score correlations (|r|) between standard PCA and FPCA:\n")
```

Score correlations (|r|) between standard PCA and FPCA:

```
for (i in 1:3) {
  r <- abs(cor(pca_raw$x[, i], fpca$x[, i]))
  cat(sprintf("  PC%d: |r| = %.4f\n", i, r))
}
```

```
PC1: |r| = 1.0000
PC2: |r| = 0.9998
PC3: |r| = 0.9997
```

The scores are identical to four decimal places — **the same wines land in the same positions on the score plot** regardless of whether you analyze the raw matrix or the Andrews curves. The variance percentages differ slightly because FPCA operates on a 200-point discretized

curve (which introduces a basis weighting), but the ranking and relative proportions are the same.

So again: why bother? Three reasons:

1. Standard PCA gives you a loading *table*; FPCA gives you eigenfunction *curves*.
Here is what PC1 looks like in each framework:

```
# Standard PCA loadings: a bar plot of 13 numbers
df_pca_loadings <- data.frame(
  Chemical = chem_names,
  Loading = pca_raw$rotation[, 1]
)

p_raw_loadings <- ggplot(df_pca_loadings,
  aes(x = reorder(Chemical, abs>Loading)),
  y = Loading, fill = Loading > 0)) +
  geom_col(alpha = 0.8, show.legend = FALSE) +
  coord_flip() +
  scale_fill_manual(values = c("TRUE" = "steelblue", "FALSE" = "coral")) +
  labs(title = "Standard PCA: PC1 Loadings",
    subtitle = "13 numbers - which chemicals contribute most?",
    x = NULL, y = "Loading") +
  theme_minimal()

# FPCA eigenfunction: a curve over [- , ]
df_eigen <- data.frame(t = t_grid, loading = fpca$rotation$data[1, ])

p_eigen <- ggplot(df_eigen, aes(x = t, y = loading)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_hline(yintercept = 0, linetype = "dashed", alpha = 0.5) +
  labs(title = "FPCA: PC1 Eigenfunction",
    subtitle = "A continuous mode of variation - peaks and troughs show where wines differ",
    x = expression(t), y = "Loading") +
  theme_minimal()

p_raw_loadings / p_eigen
```

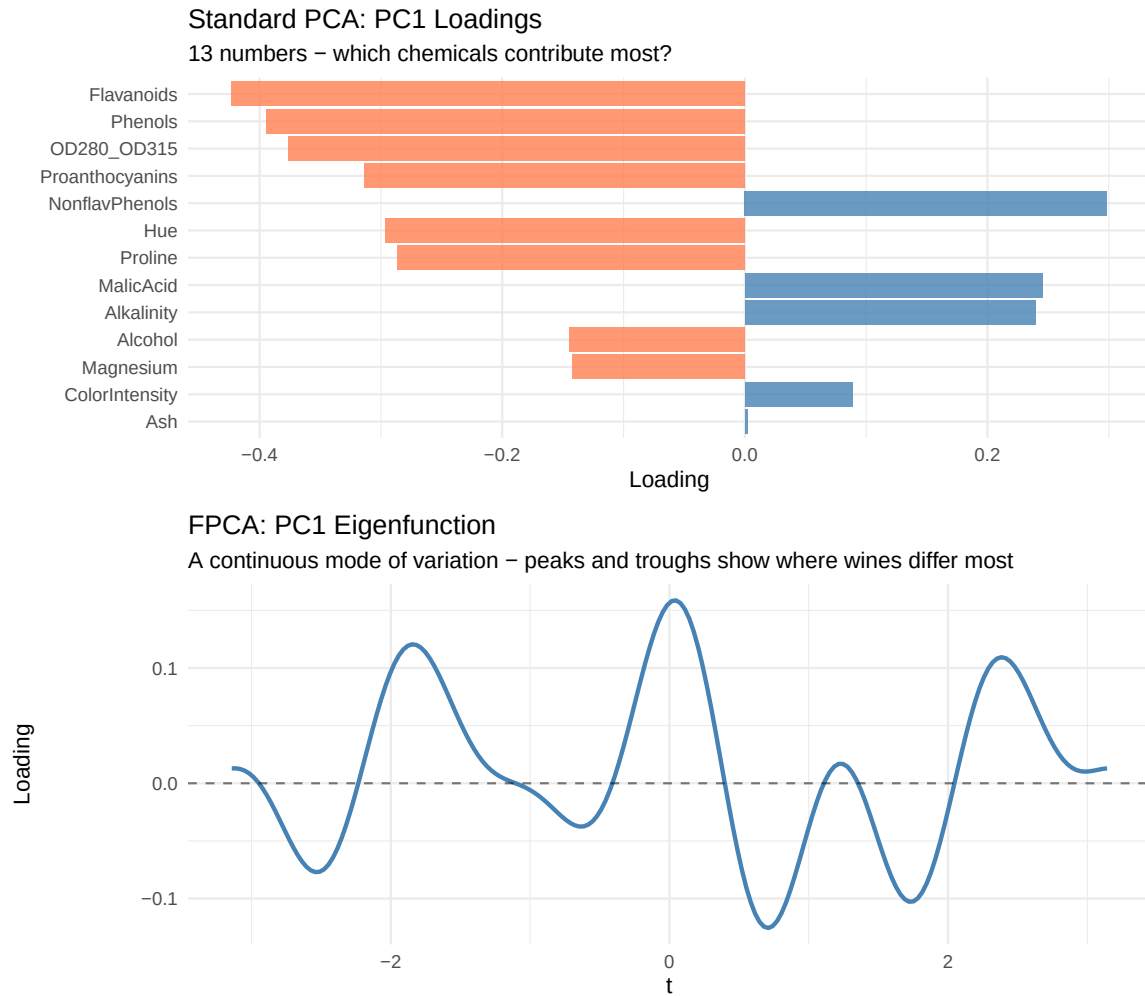


Figure 14: Standard PCA loadings (top) vs FPCA eigenfunction (bottom) — the same information, but the curve reveals the mode of variation as a visual shape.

The bar plot tells you *which variables* load heavily on PC1. The eigenfunction tells you the same thing, but as a **continuous shape** — you can see the *mode of variation* as a visual signature. When the curve peaks, the corresponding Fourier frequencies (and therefore chemicals) are driving separation. When it crosses zero, those frequencies contribute nothing. For presentations and reports, the eigenfunction communicates the structure of variability far more intuitively than a table of 13 signed numbers.

2. FPCA integrates naturally with the rest of the FDA toolkit. Once you have Andrews curves, you can chain FPCA with depth, outlier detection, clustering, bootstrap CIs, and the functional boxplot — all operating in the same functional space, all using the same

`fdata` object. Standard PCA is a standalone analysis step; FPCA is a module in a connected pipeline.

3. The projection-back step gives you variable importance *in context*. With standard PCA, you read variable loadings directly from the rotation matrix. With FPCA, you project eigenfunctions back onto the Fourier basis to recover variable importance — and those loadings are weighted by how each chemical contributes to the *curve's shape*, not just to a linear combination. The results are consistent, but the FPCA framing connects variable importance to the visual output: the chemicals that shape the eigenfunction curves are the ones that shape the Andrews curves you've been looking at throughout this analysis.

i Honest assessment

If your only goal is “run PCA and get loadings,” standard `prcomp()` is simpler and gives you the same answer. The value of FPCA emerges when you're already using Andrews curves for visualization, outlier detection, or clustering — then FPCA slots into the same pipeline rather than being a separate analysis track.

Variable Ordering

Because the first Fourier terms carry the most visual weight, **placing important variables first** produces more informative plots. We rank the 13 chemicals by ANOVA F-statistic:

```
f_stats <- sapply(1:ncol(X), function(j) {  
  fit <- aov(X[, j] ~ cultivar)  
  summary(fit)[[1]]$`F value`[1]  
})  
names(f_stats) <- chem_names  
var_order <- order(f_stats, decreasing = TRUE)  
  
cat("Variables ranked by ANOVA F-statistic:\n")
```

Variables ranked by ANOVA F-statistic:

```
for (i in var_order) {  
  cat(sprintf("  %s: F = %.1f\n", names(f_stats)[i], f_stats[i]))  
}
```

```
Flavanoids: F = 233.9  
Proline: F = 207.9  
OD280_OD315: F = 190.0
```

Alcohol: F = 135.1
 ColorIntensity: F = 120.7
 Hue: F = 101.3
 Phenols: F = 93.7
 MalicAcid: F = 36.9
 Alkalinity: F = 35.8
 Proanthocyanins: F = 30.3
 NonflavPhenols: F = 27.6
 Ash: F = 13.3
 Magnesium: F = 12.4

```

fd_original <- andrews_transform(X)
fd_reordered <- andrews_transform(X[, var_order])

# Helper to make curve data frame
make_curve_df <- function(fd, label) {
  data.frame(
    t = rep(t_grid, n),
    value = as.vector(t(fd$data)),
    curve = rep(1:n, each = m),
    Cultivar = rep(cultivar, each = m),
    Ordering = label
  )
}

p_orig <- ggplot(make_curve_df(fd_original, "Original"),
  aes(x = t, y = value, group = curve, color = Cultivar)) +
  geom_line(alpha = 0.3, linewidth = 0.35) +
  scale_color_manual(values = c("Barolo" = "#8B0000", "Grignolino" = "#DAA520",
    "Barbera" = "#2E8B57")) +
  labs(title = "Original Variable Order", x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "none")

p_reord <- ggplot(make_curve_df(fd_reordered, "Reordered"),
  aes(x = t, y = value, group = curve, color = Cultivar)) +
  geom_line(alpha = 0.3, linewidth = 0.35) +
  scale_color_manual(values = c("Barolo" = "#8B0000", "Grignolino" = "#DAA520",
    "Barbera" = "#2E8B57")) +
  labs(title = "Reordered by ANOVA F-statistic", x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

p_orig / p_reord

```

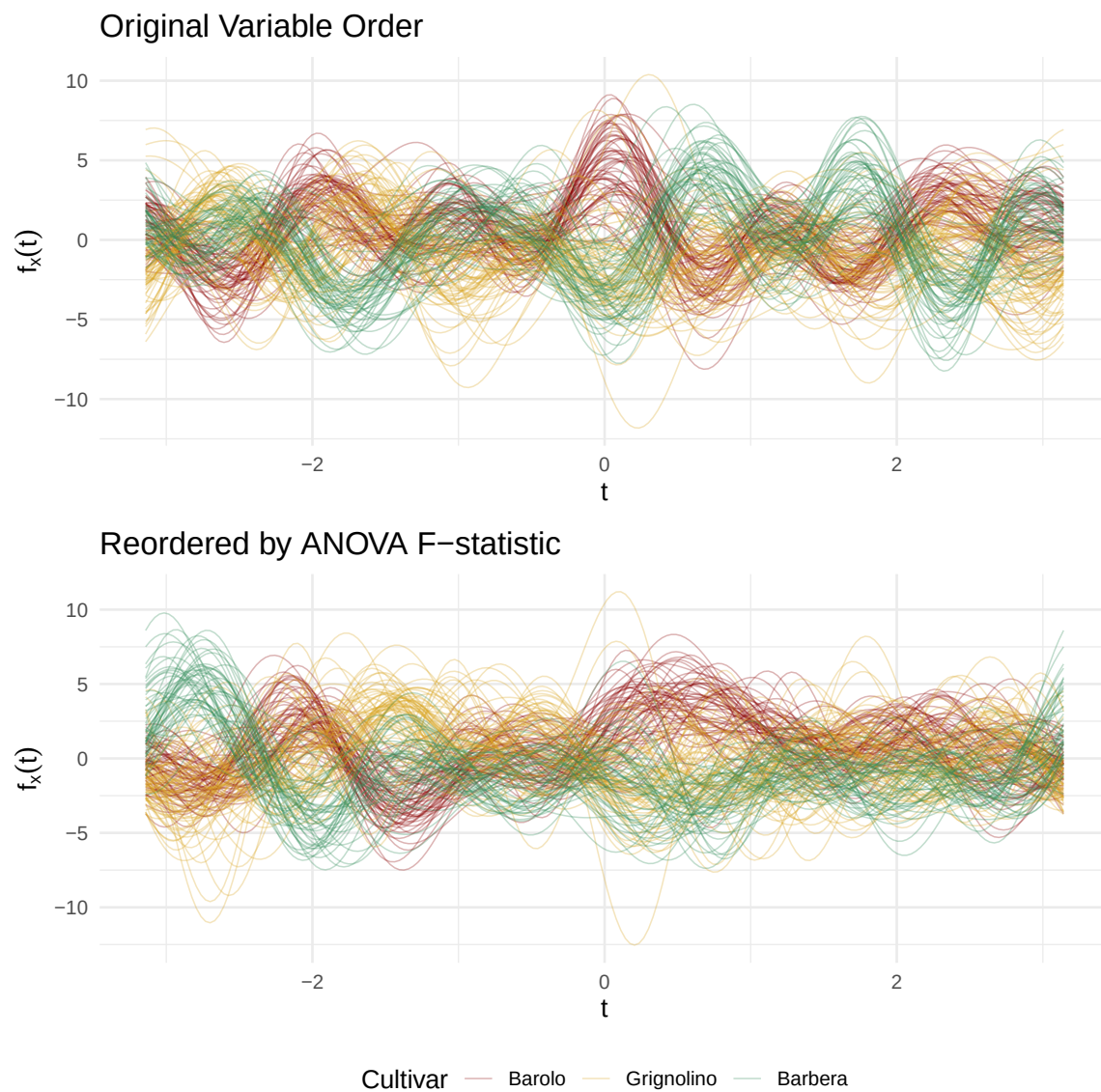



Figure 15: Variable ordering affects visual clarity but not analytical results — distance preservation holds regardless.

The reordered plot shows clearer cultivar separation because the most discriminating chemicals now drive the dominant low-frequency Fourier terms. Crucially, clustering results are **identical** regardless of ordering — this is a presentation improvement, not a data manipulation.

The Functional Boxplot: A Cultivar Specification

Classical boxplots summarize one variable at a time. If you want to monitor 13 chemicals, you need 13 boxplots, and there's no way to assess whether the *combination* of values is normal. The functional boxplot solves this: it defines a simultaneous envelope over all 13 chemicals (encoded in the curve), giving you a single visual that says “this is what a normal wine looks like.”

Overall Functional Boxplot

```
fbp <- boxplot(fd_wine)
fbp$plot
```

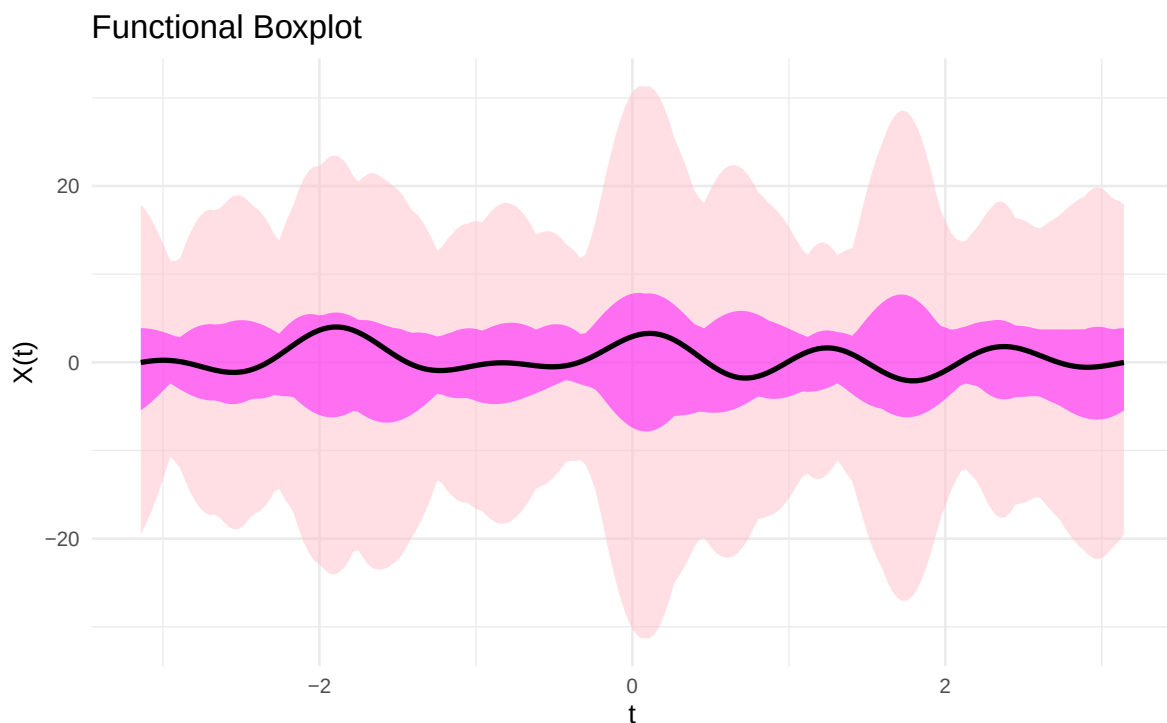


Figure 16: Functional boxplot of all 178 wines — median curve (black), 50% central region (magenta), fences (pink), and outliers (red).

The median curve (black) is the **most typical wine** — the one with the highest functional depth. The magenta band is the **50% central region**: half of all wines fall within this envelope

at every point. The pink band extends to the fences ($1.5\times$ the envelope width); any wine whose curve escapes the pink band at *any* Fourier frequency is automatically flagged.

Per-Cultivar Specifications

The overall boxplot mixes all three cultivars. For quality control, what you really want is a **cultivar-specific specification** — what should a typical Barolo, Grignolino, or Barbera look like?

```
bp_list <- list()
for (cv in levels(cultivar)) {
  idx <- which(cultivar == cv)
  fd_cv <- fd_wine[idx]
  fbp_cv <- boxplot(fd_cv)

  bp_list[[cv]] <- fbp_cv$plot +
    labs(title = sprintf("%s (n = %d)", cv, length(idx)),
         x = expression(t), y = expression(f[x](t)))
}

bp_list[["Barolo"]] / bp_list[["Grignolino"]] / bp_list[["Barbera"]]
```

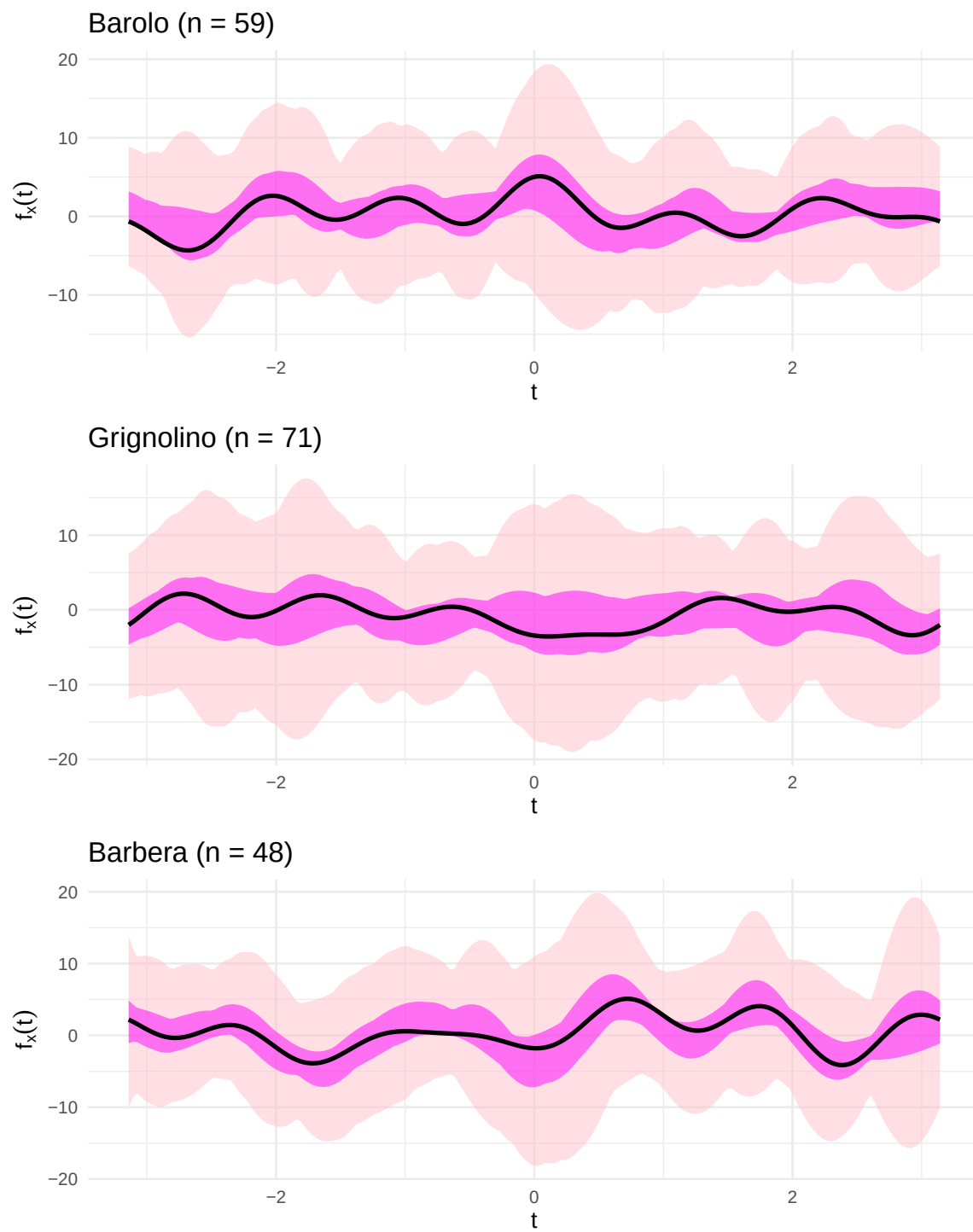


Figure 17: Per-cultivar functional boxplots — each cultivar gets its own ‘normal range’ specification.

These three boxplots define a **visual specification** for each cultivar. A quality controller can now take a new wine, compute its Andrews curve, and overlay it on the appropriate cultivar's boxplot. If the curve falls within the magenta band, the wine is typical. If it escapes the pink fences, investigate.

Incoming Batch Monitoring

Let's simulate this workflow. We take Wine 122 (the suspect Grignolino from our outlier analysis) and Wine 107 (the most typical Grignolino by depth) and check them against the Grignolino specification:

```
grig_idx <- which(cultivar == "Grignolino")
fd_grig <- fd_wine[grig_idx]
fbp_grig <- boxplot(fd_grig)

# Extract envelope and fence for custom overlay
df_envelope <- data.frame(
  t = t_grid,
  env_min = fbp_grig$envelope$min, env_max = fbp_grig$envelope$max,
  fence_min = fbp_grig$fence$min, fence_max = fbp_grig$fence$max,
  median_val = fd_grig$data[fbp_grig$median, ]
)

# Wine 122 and Wine 107 curves
df_check <- rbind(
  data.frame(t = t_grid, value = fd_wine$data[122, ], Wine = "Wine 122 (suspect)"),
  data.frame(t = t_grid, value = fd_wine$data[107, ], Wine = "Wine 107 (typical)")
)

ggplot() +
  geom_ribbon(data = df_envelope, aes(x = t, ymin = fence_min, ymax = fence_max),
    fill = "pink", alpha = 0.5) +
  geom_ribbon(data = df_envelope, aes(x = t, ymin = env_min, ymax = env_max),
    fill = "magenta", alpha = 0.4) +
  geom_line(data = df_envelope, aes(x = t, y = median_val),
    color = "black", linewidth = 1) +
  geom_line(data = filter(df_check, Wine == "Wine 107 (typical)"),
    aes(x = t, y = value), color = "steelblue", linewidth = 0.9) +
  geom_line(data = filter(df_check, Wine == "Wine 122 (suspect)"),
    aes(x = t, y = value), color = "red", linewidth = 0.9,
    linetype = "dashed") +
  labs(title = "QC Check: New Wine vs Grignolino Specification",
```

```

subtitle = "Blue = typical (inside envelope), Red dashed = suspect (outside envelope)"
x = expression(t), y = expression(f[x](t))

```

QC Check: New Wine vs Grignolino Specification

Blue = typical (inside envelope), Red dashed = suspect (outside envelope)

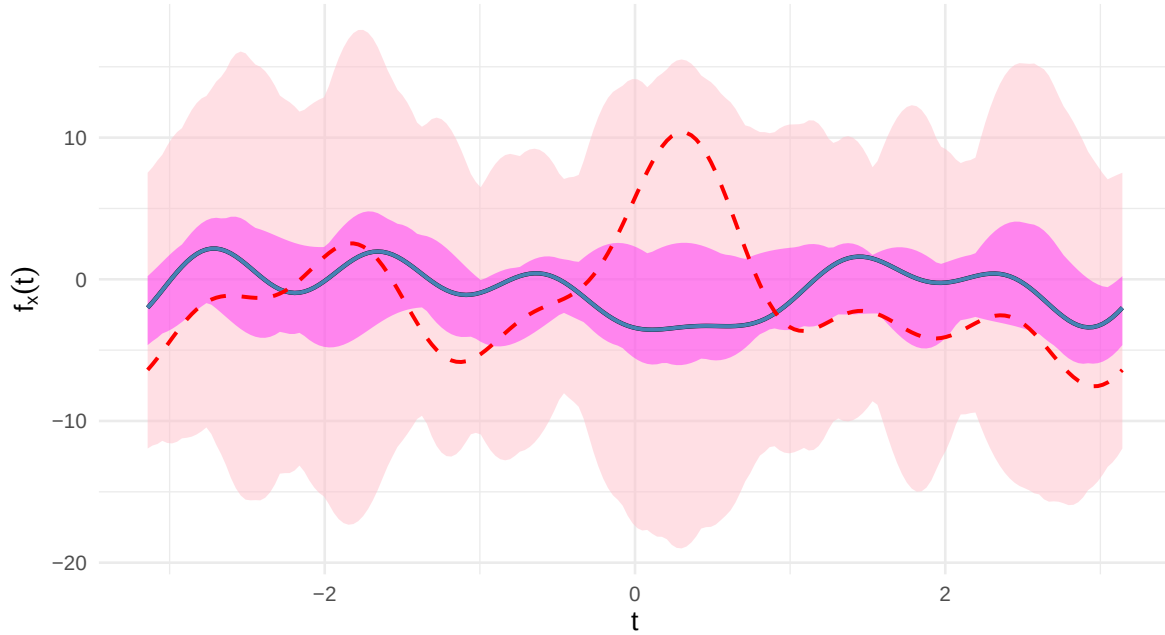


Figure 18: Incoming batch check — Wine 107 (blue) sits inside the central region; Wine 122 (red dashed) escapes it, confirming it doesn't match the Grignolino specification.

No 13-column spreadsheet comparison needed. One glance tells the story: Wine 107 traces the median closely; Wine 122's curve departs from the envelope in several regions, particularly around $t \approx 0$ where the $\sin(t)$ and $\cos(t)$ terms (encoding Malic Acid and Ash) dominate. This is a **multivariate control chart** — monitoring all 13 chemicals simultaneously in a single visual.

There is no simple classical equivalent. A 13-dimensional confidence ellipsoid exists mathematically, but you can't plot it, you can't show it to a plant manager, and you can't point to the region where the wine deviates. The functional boxplot does all three.

Depth as a Typicality Score

Functional depth gives every wine a scalar **typicality score** within its cultivar. This is a concept with no clean classical equivalent — it simultaneously accounts for all 13 chemicals

without requiring a parametric model or covariance matrix inversion.

```
# Compute within-cultivar depth rankings
depth_rankings <- lapply(levels(cultivar), function(cv) {
  idx <- which(cultivar == cv)
  fd_cv <- fd_wine[idx]
  depths <- depth.MBD(fd_cv)
  data.frame(
    Wine = idx,
    Cultivar = cv,
    Depth = round(depths, 4),
    Rank = rank(-depths)
  )
})
df_depths <- bind_rows(depth_rankings)

# Show top 3 and bottom 3 per cultivar
top_bottom <- df_depths |>
  group_by(Cultivar) |>
  arrange(Rank) |>
  slice(c(1:3, (n()-2):n())) |>
  mutate(Status = ifelse(Rank <= 3, "Most typical", "Least typical"))

kable(top_bottom |> select(Cultivar, Wine, Depth, Rank, Status),
      caption = "Most and least typical wines per cultivar (by functional depth)")
```

Table 10: Most and least typical wines per cultivar (by functional depth)

Cultivar	Wine	Depth	Rank	Status
Barbera	175	0.4756	1	Most typical
Barbera	149	0.4734	2	Most typical
Barbera	164	0.4311	3	Most typical
Barbera	153	0.2874	46	Least typical
Barbera	160	0.2434	47	Least typical
Barbera	159	0.1980	48	Least typical
Barolo	58	0.4351	1	Most typical
Barolo	49	0.4321	2	Most typical
Barolo	36	0.4084	3	Most typical
Barolo	40	0.2640	57	Least typical
Barolo	15	0.2370	58	Least typical
Barolo	26	0.1994	59	Least typical

Cultivar	Wine	Depth	Rank	Status
Grignolino	107	0.4394	1	Most typical
Grignolino	82	0.4377	2	Most typical
Grignolino	118	0.4360	3	Most typical
Grignolino	70	0.2244	69	Least typical
Grignolino	96	0.2086	70	Least typical
Grignolino	122	0.1938	71	Least typical

The “most typical” wines are the ones a sommelier would call textbook examples of their cultivar. The “least typical” wines — 122, 96, 70 for Grignolino; 159 for Barbera — are exactly the ones our outlier analysis flagged. Depth provides a continuous ranking, not a binary flag, so the quality controller can set their own threshold: investigate the bottom 5%? The bottom 10%?

Robust Location: Trimmed Mean vs Ordinary Mean

Depth also enables **robust estimation**. The trimmed mean discards the shallowest curves (the most atypical wines) before averaging, giving a center estimate that resists contamination:

```
fd_grig <- fd_wine[which(cultivar == "Grignolino")]

# Full sample estimates
fn_mean_full <- mean(fd_grig)
fn_trimmed_full <- trimmed(fd_grig, trim = 0.25)

# Remove 3 most extreme outliers and recompute
grig_clean <- setdiff(which(cultivar == "Grignolino"), c(70, 96, 122))
fd_grig_clean <- fd_wine[grig_clean]
fn_mean_clean <- mean(fd_grig_clean)
fn_trimmed_clean <- trimmed(fd_grig_clean, trim = 0.25)

# Measure how much each shifted
mean_shift <- sqrt(sum((fn_mean_full$data - fn_mean_clean$data)^2) *
  diff(t_grid[1:2]))
trimmed_shift <- sqrt(sum((fn_trimmed_full$data - fn_trimmed_clean$data)^2) *
  diff(t_grid[1:2]))

cat(sprintf("Removing 3 extreme Grignolino outliers (wines 70, 96, 122):\n"))
```

Removing 3 extreme Grignolino outliers (wines 70, 96, 122):


```
cat(sprintf(" Mean curve shifted by:          %.2f\n", mean_shift))
```

Mean curve shifted by: 0.32

```
cat(sprintf(" 25%% trimmed mean shifted by:    %.2f\n", trimmed_shift))
```

25% trimmed mean shifted by: 0.22

```
cat(sprintf(" Trimmed mean is %.0f%% more stable\n",  
          100 * (mean_shift - trimmed_shift) / mean_shift))
```

Trimmed mean is 33% more stable

```
df_robust <- rbind(  
  data.frame(t = t_grid, value = as.numeric(fn_mean_full$data),  
             Estimator = "Mean (all 71 wines)"),  
  data.frame(t = t_grid, value = as.numeric(fn_trimmed_full$data),  
             Estimator = "25% Trimmed Mean"),  
  data.frame(t = t_grid, value = as.numeric(fn_mean_clean$data),  
             Estimator = "Mean (68 wines, outliers removed)")  
)  
  
ggplot(df_robust, aes(x = t, y = value, color = Estimator, linetype = Estimator)) +  
  geom_line(linewidth = 1) +  
  scale_color_manual(values = c("Mean (all 71 wines)" = "steelblue",  
                                "25% Trimmed Mean" = "darkorange",  
                                "Mean (68 wines, outliers removed)" = "steelblue")) +  
  scale_linetype_manual(values = c("Mean (all 71 wines)" = "solid",  
                                   "25% Trimmed Mean" = "solid",  
                                   "Mean (68 wines, outliers removed)" = "dashed")) +  
  labs(title = "Grignolino: Mean vs Trimmed Mean",  
       subtitle = "Dashed blue = mean after removing outliers; solid orange = trimmed mean W",  
       x = expression(t), y = expression(bar(f)(t))) +  
  theme(legend.position = "bottom",  
        legend.title = element_blank())
```

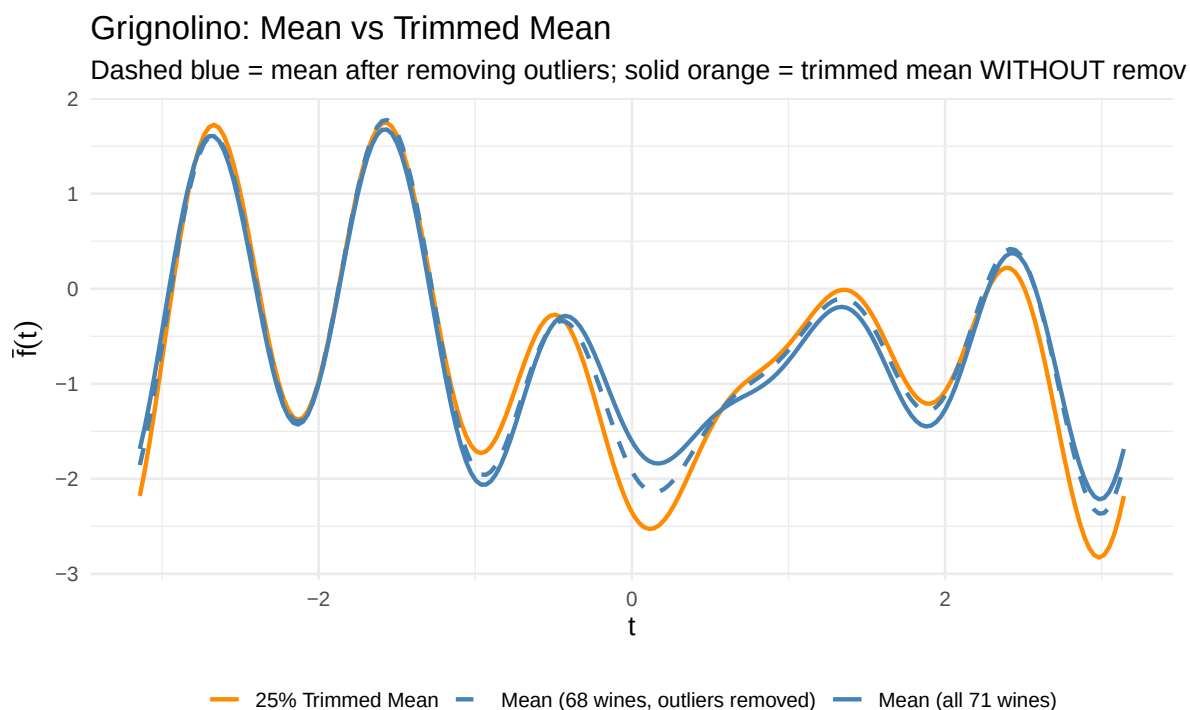


Figure 19: The 25% trimmed mean (orange) is more stable than the ordinary mean (blue) when outliers are present.

The trimmed mean (orange) computed on **all** 71 Grignolinos is close to the ordinary mean computed after manually removing outliers (dashed blue). The trimmed mean achieves robustness automatically — no need to first detect outliers and then decide which to remove. For a production pipeline where you can't manually curate every batch, this is a significant practical advantage.

From Validation to Monitoring

So far, this article has been a retrospective analysis: we took 178 wines, found outliers, verified clusters, identified important chemicals. But a quality lab doesn't analyze the same 178 wines forever. New batches arrive every week. The real question is: **can we turn this analysis into an ongoing monitoring system?**

The answer is yes, and the functional framework makes it natural. The workflow has three phases.

Phase 1: Establish the Reference (Process Validation)

During process validation, you collect a representative set of wines for each cultivar and compute the **reference profile**: the trimmed mean curve and an envelope that captures the expected range of normal variation.

We use the bootstrap to construct two types of bands:

- A **confidence band** (blue) for the trimmed mean — this quantifies uncertainty about the cultivar’s *average* profile (“where is the center?”).
- A **prediction band** (gray) for individual wines — this captures the range where a *single new observation* is expected to fall (“is this wine normal?”).

Both are **prediction intervals**: they use the observed sample to predict where future observations or statistics will land. This is the appropriate tool for *routine monitoring*, where you ask “does this new wine look like my reference set?”

For *process transfer* — validating that a new vineyard, lab, or fermentation facility produces equivalent wines — the statistically correct tool is a **tolerance interval**. A tolerance interval guarantees that *at least a specified proportion* of the population (e.g., 95%) falls within the band, with a given confidence level (e.g., 99%). This dual coverage is stricter than a prediction interval, which only covers the *next single observation*. Computing functional tolerance intervals requires adjustments for simultaneous coverage across the entire curve domain, which goes beyond what we demonstrate here, but the conceptual framework is the same: replace the bootstrap quantiles below with the appropriate tolerance factors (e.g., based on non-central χ^2 or Bonferroni-corrected normal order statistics). For the purposes of this article, we proceed with prediction bands as a practical approximation:

```
tolerance_plots <- list()

for (cv in levels(cultivar)) {
  idx <- which(cultivar == cv)
  fd_cv <- fd_wine[idx]

  # Bootstrap the trimmed mean to get a tolerance band
  # Using the percentile method on pointwise quantiles of bootstrap curves
  n_boot <- 500
  boot_obj <- fdata.bootstrap(fd_cv, n.boot = n_boot, method = "naive",
                             seed = 42)

  # For each bootstrap sample, compute the trimmed mean
  boot_means <- matrix(NA, n_boot, m)
  for (b in seq_len(n_boot)) {
    boot_means[b, ] <- as.numeric(trimmed(boot_obj$boot.samples[[b]]),
```

```

trim = 0.25)$data)
}

# Pointwise 2.5% and 97.5% quantiles of bootstrap trimmed means
band_lower <- apply(boot_means, 2, quantile, probs = 0.025)
band_upper <- apply(boot_means, 2, quantile, probs = 0.975)

# Also compute a wider prediction band using the raw bootstrap curves
# This captures individual wine variability, not just mean uncertainty
boot_curves <- matrix(NA, n_boot, m)
for (b in seq_len(n_boot)) {
  # Pick a random curve from each bootstrap sample
  boot_curves[b, ] <- boot_obj$boot.samples[[b]]$data[
    sample(nrow(boot_obj$boot.samples[[b]]$data), 1), ]
}
pred_lower <- apply(boot_curves, 2, quantile, probs = 0.025)
pred_upper <- apply(boot_curves, 2, quantile, probs = 0.975)

ref_mean <- as.numeric(trimmed(fd_cv, trim = 0.25)$data)

df_band <- data.frame(
  t = t_grid,
  mean = ref_mean,
  ci_lower = band_lower, ci_upper = band_upper,
  pred_lower = pred_lower, pred_upper = pred_upper
)

tolerance_plots[[cv]] <- ggplot(df_band, aes(x = t)) +
  geom_ribbon(aes(ymin = pred_lower, ymax = pred_upper),
    fill = "gray80", alpha = 0.6) +
  geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper),
    fill = "steelblue", alpha = 0.3) +
  geom_line(aes(y = mean), color = "navy", linewidth = 1) +
  labs(title = sprintf("%s (n = %d)", cv, length(idx)),
    subtitle = "Navy = trimmed mean, Blue = 95% CI of mean, Gray = 95% prediction band",
    x = expression(t), y = expression(f[x](t))) +
  theme_minimal()
}

tolerance_plots[["Barolo"]] / tolerance_plots[["Grignolino"]] / tolerance_plots[["Barbera"]]

```

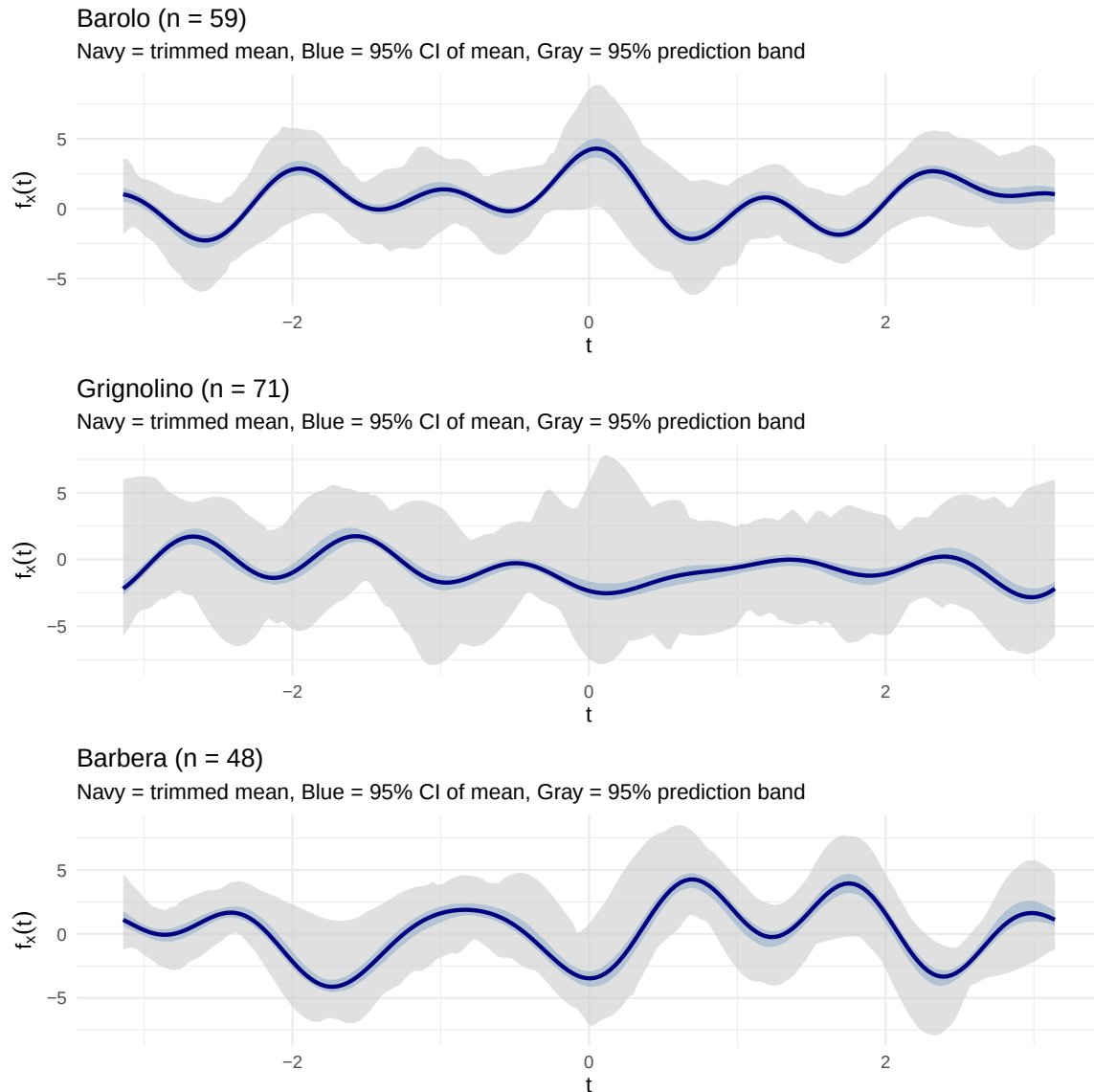


Figure 20: 95% bootstrap tolerance bands per cultivar — a new wine whose curve stays inside the band (gray) passes the QC check.

The **blue band** (confidence band for the mean) answers: “where does the cultivar’s average profile sit?” The **gray band** (prediction band for individual wines) answers: “where can a single new wine’s curve fall and still be considered normal?” For routine QC, the gray prediction band is the operational acceptance criterion: any new wine whose Andrews curve stays entirely inside it passes.

For formal process transfer validation, you would replace the gray band with a proper tolerance band (wider, to guarantee population coverage with confidence). The prediction band shown here is a pragmatic starting point that works well for ongoing monitoring.

Phase 2: Process Transfer

When a process is transferred — to a new vineyard, a new fermentation facility, or a new analytical lab — you need to verify that the new process produces wines within the validated specification. Ideally this would use tolerance bands (guaranteeing population coverage); here we demonstrate the workflow with prediction bands as a practical approximation.

The key advantage of the functional approach is that when a wine *fails*, you can trace back to **which chemicals** caused the deviation. Because the Andrews curve maps each Fourier frequency to specific input variables, a violation at a particular region of the curve implicates specific chemicals. We can also directly compare the original (standardized) chemical values to pinpoint exactly where the new wine deviates:

```
# Simulate: take a Barolo wine and check against the Barolo tolerance band
barolo_idx <- which(cultivar == "Barolo")
fd_barolo <- fd_wine[barolo_idx]

# Compute the prediction band for Barolo
boot_barolo <- fdata.bootstrap(fd_barolo, n.boot = 500, method = "naive",
                              seed = 42)
boot_barolo_curves <- matrix(NA, 500, m)
for (b in 1:500) {
  boot_barolo_curves[b, ] <- boot_barolo$boot.samples[[b]]$data[
    sample(nrow(boot_barolo$boot.samples[[b]]$data), 1), ]
}
barolo_pred_lower <- apply(boot_barolo_curves, 2, quantile, probs = 0.025)
barolo_pred_upper <- apply(boot_barolo_curves, 2, quantile, probs = 0.975)
barolo_mean <- as.numeric(trimmed(fd_barolo, trim = 0.25)$data)

df_barolo_band <- data.frame(
  t = t_grid,
  mean = barolo_mean,
  pred_lower = barolo_pred_lower,
  pred_upper = barolo_pred_upper
)

# Test wines: a true Barolo (wine 36, typical) and a Grignolino (wine 107)
df_test_wines <- rbind(
```

```

data.frame(t = t_grid, value = fd_wine$data[36, ],
           Wine = "Wine 36 (Barolo - typical)",
data.frame(t = t_grid, value = fd_wine$data[107, ],
           Wine = "Wine 107 (Grignolino)")
)

# Check which wines stay inside the band
for (w in c(36, 107)) {
  inside <- all(fd_wine$data[w, ] >= barolo_pred_lower &
               fd_wine$data[w, ] <= barolo_pred_upper)
  cat(sprintf("Wine %d (%s): %s Barolo prediction band\n",
             w, cultivar[w], ifelse(inside, "INSIDE", "OUTSIDE")))
}

```

Wine 36 (Barolo): INSIDE Barolo prediction band
Wine 107 (Grignolino): OUTSIDE Barolo prediction band

```

ggplot() +
  geom_ribbon(data = df_barolo_band,
            aes(x = t, ymin = pred_lower, ymax = pred_upper),
            fill = "gray80", alpha = 0.6) +
  geom_line(data = df_barolo_band, aes(x = t, y = mean),
            color = "navy", linewidth = 1) +
  geom_line(data = filter(df_test_wines, Wine == "Wine 36 (Barolo - typical)",
            aes(x = t, y = value), color = "#2E8B57", linewidth = 0.9) +
  geom_line(data = filter(df_test_wines, Wine == "Wine 107 (Grignolino)",
            aes(x = t, y = value), color = "red", linewidth = 0.9,
            linetype = "dashed") +
  labs(title = "Process Transfer Check: Barolo Prediction Band",
       subtitle = "Green = true Barolo (passes), Red dashed = Grignolino (fails)",
       x = expression(t), y = expression(f[x](t)))

```

Process Transfer Check: Barolo Prediction Band

Green = true Barolo (passes), Red dashed = Grignolino (fails)

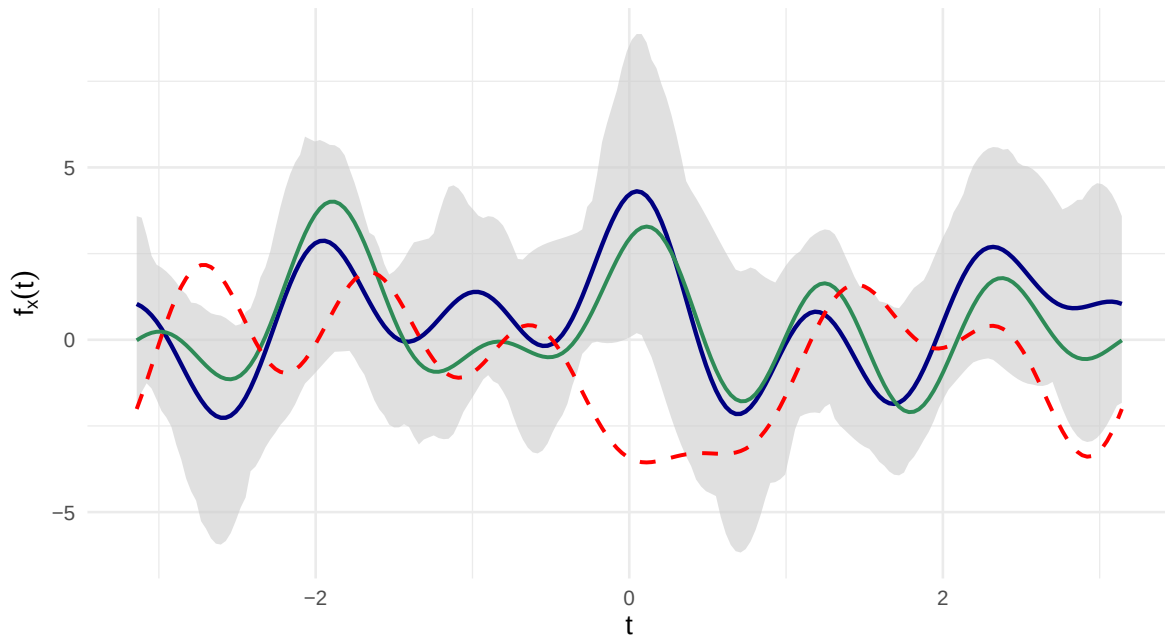


Figure 21: Process transfer check — does a new wine from a different source match the Barolo specification?

A true Barolo stays inside the gray band; a Grignolino immediately violates it. But the critical follow-up question is: **which chemicals caused the failure?** Because we preserved the mapping between curve and original variables, we can answer this directly:

```
# Compare wine 107 (Grignolino) against Barolo reference ranges
barolo_means <- colMeans(X[barolo_idx, ])
barolo_sds <- apply(X[barolo_idx, ], 2, sd)

wine107_z <- (X[107, ] - barolo_means) / barolo_sds

df_diag <- data.frame(
  Chemical = chem_names,
  z_score = as.numeric(wine107_z)
) |>
dplyr::mutate(
  Status = dplyr::case_when(
    abs(z_score) > 2 ~ "Out of spec (|z| > 2)",
    abs(z_score) > 1 ~ "Borderline (1 < |z| < 2)",
```



```

    TRUE ~ "Within spec ( $|z| < 1$ )"
  ),
  Chemical = factor(Chemical, levels = Chemical[order(abs(z_score))])
)

ggplot(df_diag, aes(x = Chemical, y = z_score, fill = Status)) +
  geom_col() +
  geom_hline(yintercept = c(-2, 2), linetype = "dashed", color = "red") +
  geom_hline(yintercept = c(-1, 1), linetype = "dotted", color = "orange") +
  coord_flip() +
  scale_fill_manual(values = c("Out of spec ( $|z| > 2$ )" = "red",
                                "Borderline ( $1 < |z| < 2$ )" = "orange",
                                "Within spec ( $|z| < 1$ )" = "#2E8B57")) +
  labs(title = "Wine 107 vs Barolo Specification",
       subtitle = "z-scores relative to Barolo mean and SD",
       x = NULL, y = "Standard deviations from Barolo mean") +
  theme(legend.position = "bottom")

```

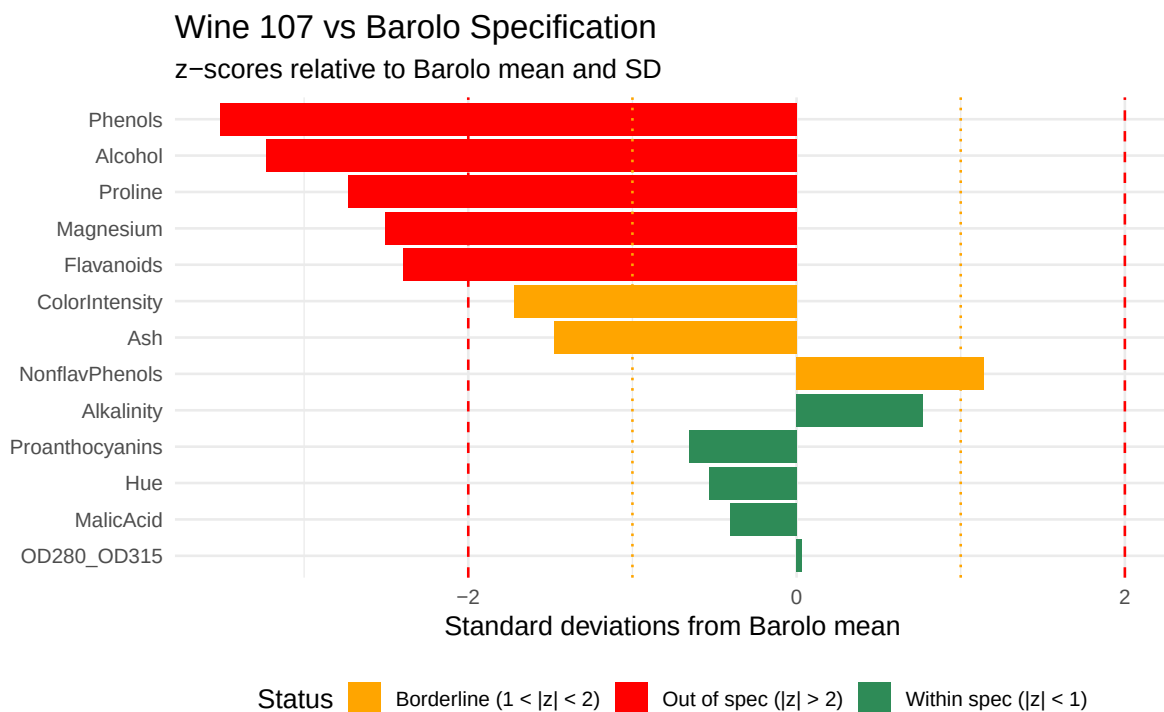


Figure 22: Chemical-level diagnostic: which measurements caused Wine 107 to fail the Barolo specification?

```
# Report the failing chemicals
failing <- df_diag |> dplyr::filter(abs(z_score) > 2) |>
  dplyr::arrange(dplyr::desc(abs(z_score)))
cat("Chemicals out of spec (|z| > 2):\n")
```

Chemicals out of spec (|z| > 2):

```
for (i in seq_len(nrow(failing))) {
  cat(sprintf(" %s: z = %+1f (%s Barolo range)\n",
    failing$Chemical[i], failing$z_score[i],
    ifelse(failing$z_score[i] > 0, "above", "below")))
}
```

```
Phenols: z = -3.5 (below Barolo range)
Alcohol: z = -3.2 (below Barolo range)
Proline: z = -2.7 (below Barolo range)
Magnesium: z = -2.5 (below Barolo range)
Flavanoids: z = -2.4 (below Barolo range)
```

This is the operational payoff: not just “this wine fails the Barolo check” but “it fails *because* its Flavanoids are too low and its Color Intensity is too high” — actionable information that tells the lab exactly what to re-test or the quality manager what went wrong in production.

Phase 3: Ongoing Process Monitoring

Once the process is validated, monitoring becomes routine. Each new batch is:

1. Measured for the 13 chemicals (or the reduced set identified by FPCA)
2. Transformed into an Andrews curve
3. Checked against the cultivar-specific tolerance band *and* the functional boxplot fences

```
# Build the monitoring view: functional boxplot + incoming wines
grig_idx <- which(cultivar == "Grignolino")
fd_grig <- fd_wine[grig_idx]
fbp_grig <- boxplot(fd_grig)

df_monitor <- data.frame(
  t = t_grid,
  env_min = fbp_grig$envelope$min, env_max = fbp_grig$envelope$max,
  fence_min = fbp_grig$fence$min, fence_max = fbp_grig$fence$max,
```

```

    median_val = fd_grig$data[fbp_grig$median, ]
  )

# "Incoming" wines: one normal, one outlier, one mislabeled
incoming <- rbind(
  data.frame(t = t_grid, value = fd_wine$data[82, ],
             Wine = "Batch A (normal)", Status = "Pass"),
  data.frame(t = t_grid, value = fd_wine$data[96, ],
             Wine = "Batch B (Mg outlier)", Status = "Investigate"),
  data.frame(t = t_grid, value = fd_wine$data[122, ],
             Wine = "Batch C (mislabel?)", Status = "Reject")
)

ggplot() +
  geom_ribbon(data = df_monitor, aes(x = t, ymin = fence_min, ymax = fence_max),
            fill = "pink", alpha = 0.4) +
  geom_ribbon(data = df_monitor, aes(x = t, ymin = env_min, ymax = env_max),
            fill = "magenta", alpha = 0.3) +
  geom_line(data = df_monitor, aes(x = t, y = median_val),
            color = "black", linewidth = 0.8) +
  geom_line(data = incoming,
            aes(x = t, y = value, color = Status, linetype = Status),
            linewidth = 0.8) +
  scale_color_manual(values = c("Pass" = "#2E8B57", "Investigate" = "orange",
                                "Reject" = "red")) +
  scale_linetype_manual(values = c("Pass" = "solid", "Investigate" = "dashed",
                                    "Reject" = "dotted")) +
  labs(title = "Grignolino Process Monitor",
       subtitle = "Green = passes; Orange dashed = outside central region (investigate); Red",
       x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

Grignolino Process Monitor

Green = passes; Orange dashed = outside central region (investigate); Red dotted = outside

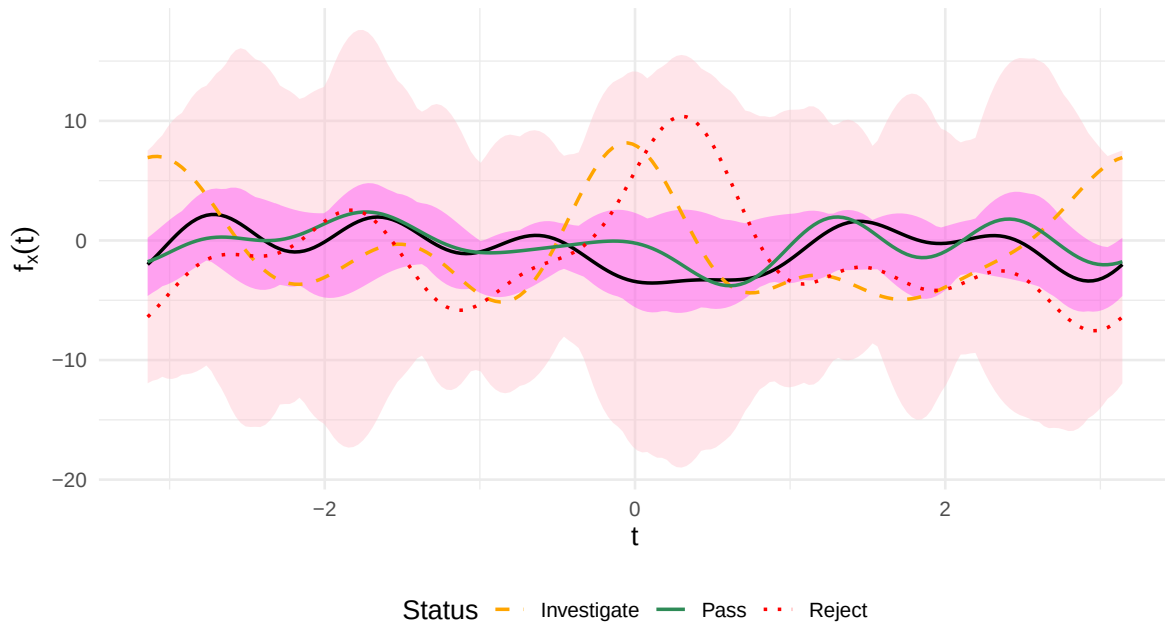


Figure 23: Process monitoring dashboard — new wines plotted against the functional boxplot. Inside the magenta band = normal; outside pink fences = investigate.

Again, the crucial advantage is that a monitoring alert doesn't end with "investigate" — it tells you **which chemicals** are driving the deviation. For each flagged batch, we can decompose the alert back into the original measurement space:

```
# For the flagged wines (96 = investigate, 122 = reject), compute z-scores
# relative to Grignolino reference
grig_means <- colMeans(X[grig_idx, ])
grig_sds <- apply(X[grig_idx, ], 2, sd)

diag_list <- list()
for (w in c(96, 122)) {
  wz <- (X[w, ] - grig_means) / grig_sds
  diag_list[[as.character(w)]] <- data.frame(
    Chemical = chem_names,
    z_score = as.numeric(wz),
    Wine = sprintf("Wine %d (%s)", w, cultivar[w])
  )
}
```

```

df_mon_diag <- do.call(rbind, diag_list) |>
  dplyr::mutate(
    Status = dplyr::case_when(
      abs(z_score) > 2 ~ "Out of spec",
      abs(z_score) > 1 ~ "Borderline",
      TRUE ~ "Within spec"
    )
  )

ggplot(df_mon_diag, aes(x = Chemical, y = z_score, fill = Status)) +
  geom_col() +
  geom_hline(yintercept = c(-2, 2), linetype = "dashed", color = "red") +
  coord_flip() +
  scale_fill_manual(values = c("Out of spec" = "red",
                                "Borderline" = "orange",
                                "Within spec" = "#2E8B57")) +

  facet_wrap(~ Wine) +
  labs(title = "Chemical-Level Diagnostic for Flagged Batches",
       subtitle = "z-scores relative to Grignolino mean and SD",
       x = NULL, y = "Standard deviations from cultivar mean") +
  theme(legend.position = "bottom")

```

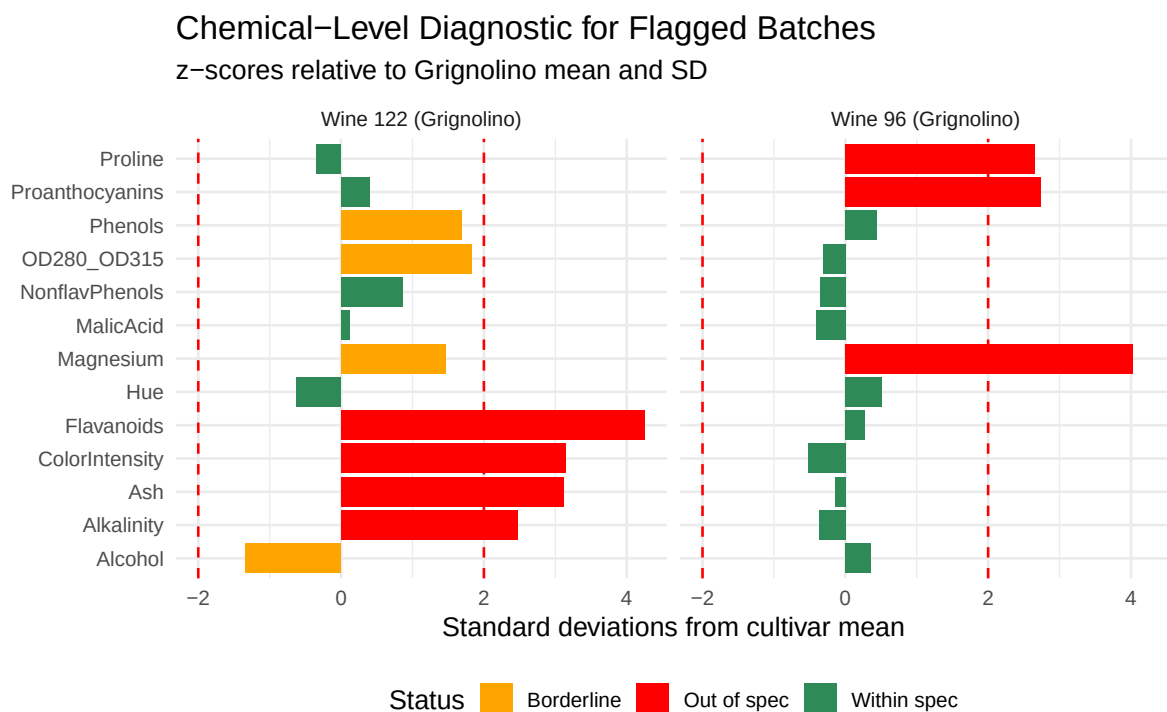


Figure 24: Monitoring diagnostic: which chemicals drive each alert?

Wine 96 (the “Investigate” batch) has a small number of chemicals pulling it outside the central region — the alert is localized, and the quality team knows exactly which assays to re-run. Wine 122 (the “Reject” batch) deviates on a broad set of chemicals in a pattern consistent with a different cultivar entirely — confirming the mislabeling hypothesis from the outlier analysis.

This is the operational payoff of the entire pipeline. The validation phase (sections 2–8 of this article) produces the reference profiles, prediction bands, and decision rules. The monitoring phase applies them to every new batch with a single curve overlay *plus* a chemical-level breakdown when something goes wrong. The quality controller doesn’t need to understand Fourier analysis or functional depth — they see a curve, a band, a red/yellow/green signal, and a bar chart telling them which chemicals to investigate.

For regulated industries, this workflow maps directly to the **process validation lifecycle** (ICH Q8-Q12, FDA guidance):

- **Stage 1 (Process Design):** FPCA identifies critical quality attributes (which chemicals matter); clustering validates groupings

- **Stage 2 (Process Qualification):** prediction bands define acceptance criteria (tolerance bands for formal validation); the permutation test provides statistical evidence of cultivar distinctness
- **Stage 3 (Continued Process Verification):** the functional boxplot serves as the ongoing control chart; depth ranking flags drifting batches before they exceed limits; chemical-level diagnostics pinpoint root causes

So Why Not Just Use Standard Statistics?

Throughout this article, we've compared FDA methods against their classical counterparts. Let's be honest about what we found:

```
comp_df <- data.frame(
  Task = c("Outlier detection", "Clustering", "PCA / FPCA",
           "Quality monitoring", "Hypothesis testing", "Robust estimation"),
  `Classical Method` = c(
    "Mahalanobis distance",
    "k-means on raw data",
    "prcomp() on raw matrix",
    "Per-variable control charts (13 separate charts)",
    "MANOVA (assumes multivariate normality)",
    "Trimmed column means (per variable)"
  ),
  `What FDA Adds` = c(
    "Three methods classifying the TYPE of outlier (magnitude vs shape vs rank); visual curves",
    "Cluster centers as curves; fuzzy membership; visual overlay",
    "Eigenfunction curves showing modes of variation; pipeline integration",
    "Functional boxplot: one chart monitoring all 13 chemicals simultaneously",
    "Nonparametric permutation test; bootstrap CIs showing WHERE profiles diverge",
    "Depth-based trimmed mean: robust center that respects multivariate structure"
  ),
  check.names = FALSE
)
kable(comp_df, caption = "Classical vs FDA: what each approach delivers")
```

Table 11: Classical vs FDA: what each approach delivers

Task	Classical Method	What FDA Adds
Outlier detection	Mahalanobis distance	Three methods classifying the TYPE of outlier (magnitude vs shape vs rank); visual curve diagnostics
Cluster- ing	k-means on raw data	Cluster centers as curves; fuzzy membership; visual overlay
PCA / FPCA	prcomp() on raw matrix	Eigenfunction curves showing modes of variation; pipeline integration
Quality monitor- ing	Per-variable control charts (13 separate charts)	Functional boxplot: one chart monitoring all 13 chemicals simultaneously
Hypothe- sis testing	MANOVA (assumes multivariate normality)	Nonparametric permutation test; bootstrap CIs showing WHERE profiles diverge
Robust estima- tion	Trimmed column means (per variable)	Depth-based trimmed mean: robust center that respects multivariate structure

The distance preservation theorem guarantees that FDA on Andrews curves produces **numerically equivalent results** to classical multivariate methods for tasks like clustering and PCA. You won't find a cluster that one approach discovers and the other misses.

But equivalence in *numerical results* doesn't mean equivalence in *analytical capability*. The FDA framework provides genuine advantages that go beyond visualization:

1. **Simultaneous multivariate monitoring.** The functional boxplot monitors all 13 chemicals in one chart. Classical quality control requires 13 separate control charts — and a wine that's within range on all 13 individual charts can still be a multivariate outlier (an unusual *combination*). The functional boxplot catches these because the curve encodes all variables simultaneously.
2. **Typed outlier diagnostics.** Mahalanobis distance gives you a number. FDA gives you a triage: *magnitude* outlier (re-test), *shape* outlier (investigate the process), or *cultivar mismatch* (audit the label). The case studies above showed this isn't abstract — Wine 122's mislabeling signal and Wine 159's concentration pattern are qualitatively different findings that require different actions.
3. **Nonparametric hypothesis testing.** The `group.test()` permutation test evaluates whether entire chemical profiles differ across cultivars, without assuming multivariate normality. Classical MANOVA gives a p-value too — but the functional bootstrap CIs show *where on the profile* the cultivars diverge, which directly identifies which chemicals to test for denomination-of-origin verification.

4. **Depth-based robust statistics.** The trimmed mean, functional median, and depth ranking all operate on the multivariate structure directly, without requiring covariance matrix inversion (which is numerically unstable when p is large relative to n and fails entirely when $p > n$). For 13 chemicals and 178 wines this isn't critical, but for higher-dimensional chemical panels (e.g., metabolomics with hundreds of features) it becomes essential.
5. **Pipeline coherence.** Every analysis step — depth, outliers, clustering, testing, FPCA, boxplot — operates on the same `fdata` object with consistent L^2 distance semantics. Classical analysis requires stitching together `mahalanobis()`, `kmeans()`, `prcomp()`, `manova()`, and ad hoc control charts, each with its own distance assumptions.
6. **Communication.** A 13-dimensional observation has no natural visual form. An Andrews curve does. Cluster centers, outlier cases, confidence bands, and cultivar specifications all become curves that a plant manager, a regulator, or a winemaker can *see* — without statistical training.

The case *against* is equally clear: if you only need one specific analysis (just PCA, or just k-means), the classical tool is simpler and gives you the same numerical answer. Andrews curves + FDA pay off when you want the **full pipeline** — from initial visualization through monitoring, outlier triage, clustering, hypothesis testing, and variable importance — in a single, coherent framework.

Conclusion

We started with a spreadsheet nobody could read and ended with an auditable analytical pipeline. Here's what each FDA method delivers:

```
summary_df <- data.frame(
  Method = c("Andrews Transform", "metric.lp()", "outliers.depth.pond()",
             "magnitudeshape()", "outliergram()",
             "cluster.kmeans()", "cluster.fcm()",
             "group.test()", "fdata.bootstrap.ci()",
             "fdata2pc()", "boxplot()"),
  `What It Reveals` = c(
    "13-dim → curve fingerprint",
    "Lossless distance ( $\sqrt{\text{factor}}$ )",
    "Low-depth anomalies",
    "Magnitude vs shape outliers",
    "Shape outliers (MBD-MEI)",
    "Unsupervised cultivar recovery",
    "Membership uncertainty",
    "Cultivar mean differences (p-value)",
```

```

    "Where on the profile cultivars diverge",
    "Dominant variation directions",
    "Central region + fences"
  ),
  `Business Decision` = c(
    "Holistic wine comparison",
    "Results translate to chemical units",
    "Flag anomalous batches",
    "Re-test sample vs investigate vineyard",
    "Detect production process anomalies",
    "Verify cultivar authenticity",
    "Identify borderline / blended wines",
    "Support denomination-of-origin claims",
    "Choose which chemicals to test",
    "Reduce lab costs (13 → 5 assays)",
    "Quality control dashboard"
  ),
  check.names = FALSE
)

kable(summary_df, caption = "FDA Pipeline: From Chemistry to Decisions")

```

Table 12: FDA Pipeline: From Chemistry to Decisions

Method	What It Reveals	Business Decision
Andrews Transform	13-dim → curve fingerprint	Holistic wine comparison
metric.lp()	Lossless distance ($\sqrt{\cdot}$ factor)	Results translate to chemical units
outliers.depth.pond()	Low-depth anomalies	Flag anomalous batches
magnitudeshape()	Magnitude vs shape outliers	Re-test sample vs investigate vineyard
outliergram()	Shape outliers (MBD-MEI)	Detect production process anomalies
cluster.kmeans()	Unsupervised cultivar recovery	Verify cultivar authenticity
cluster.fcm()	Membership uncertainty	Identify borderline / blended wines
group.test()	Cultivar mean differences (p-value)	Support denomination-of-origin claims
fdata.bootstrap.ci()	Where on the profile cultivars diverge	Choose which chemicals to test
fdata2pc()	Dominant variation directions	Reduce lab costs (13 → 5 assays)
boxplot()	Central region + fences	Quality control dashboard

Andrews curves + FDA give you a principled, auditable pipeline from raw chemistry to actionable decisions: flag anomalies, verify labels, prioritize testing, and quantify uncertainty. The **fdars** package makes this accessible from R with a Rust-accelerated backend for production-scale performance.

Built with [fdars](#) and [Quarto](#).

References

- Andrews, D.F. (1972). Plots of high-dimensional data. *Biometrics*, 28(1), 125–136.
- Arribas-Gil, A. and Romo, J. (2014). Shape outlier detection and visualization for functional data: the outliergram. *Biostatistics*, 15(4), 603–619.
- Dai, W. and Genton, M.G. (2018). Multivariate functional data visualization and outlier detection. *Journal of Computational and Graphical Statistics*, 27(4), 923–934.
- Ramsay, J.O. and Silverman, B.W. (2005). *Functional Data Analysis*. Springer.